

INFORMATIQUE • 1ÈRE E

MODULE 1 : ENVIRONNEMENT NUMÉRIQUE, SÉCURITÉ, MULTIMÉDIA ET GESTION DES DONNÉES

LEÇON N06

BASES DE DONNÉES RELATIONNELLES : INITIATION

DURÉE CONSEILLÉE : 6 H

DANS CETTE LEÇON

Cette leçon initie les élèves aux bases de données relationnelles : notions de données, tables, champs, enregistrements et clés. Elle présente les SGBD, leurs fonctions de définition et de manipulation, ainsi que les contraintes d'intégrité qui garantissent la fiabilité des données.

- À la fin de cette leçon, je sais définir les notions de donnée, base de données, SGBD, table, champ et enregistrement.
- À la fin de cette leçon, je sais décrire les composants d'un SGBD.
- À la fin de cette leçon, je sais énumérer quelques exemples de SGBD et des domaines d'application des bases de données.
- À la fin de cette leçon, je sais décrire les éléments caractéristiques d'une table : champs, enregistrements, clé primaire, clé étrangère.
- À la fin de cette leçon, je sais décrire les fonctions de définition d'une base de données (création/suppression de BD et de tables, ajout/renommage de colonnes, modification de type).
- À la fin de cette leçon, je sais décrire les fonctions de manipulation d'une base de données (insertion, mise à jour, suppression, consultation).

**TÉLÉCHARGE
ONBUCH**

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats

**PASSE À ONBUCH+**

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à
pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir

**Signé • OnBuch+**

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

SOMMAIRE

- | | |
|--|----------------------------------|
| 1. Données, bases de données et SGBD | 6. Contraintes d'intégrité |
| 2. Composants d'un SGBD | 7. Activité d'intégration |
| 3. Tables, champs, enregistrements et clés | 8. Méthodes et exercices résolus |
| 4. Fonctions de définition d'une base de données | 9. Exercices |
| 5. Fonctions de manipulation d'une base de données | 10. Corrigés des exercices |
| | 11. Fiche bilan |

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais définir les notions de donnée, base de données, SGBD, table, champ et enregistrement.
- À la fin de cette leçon, je sais décrire les composants d'un SGBD.
- À la fin de cette leçon, je sais énumérer quelques exemples de SGBD et des domaines d'application des bases de données.
- À la fin de cette leçon, je sais décrire les éléments caractéristiques d'une table : champs, enregistrements, clé primaire, clé étrangère.
- À la fin de cette leçon, je sais décrire les fonctions de définition d'une base de données (création/suppression de BD et de tables, ajout/renommage de colonnes, modification de type).
- À la fin de cette leçon, je sais décrire les fonctions de manipulation d'une base de données (insertion, mise à jour, suppression, consultation).
- À la fin de cette leçon, je sais décrire les contraintes d'intégrité : clé primaire, valeur autorisée, valeur nulle, valeur par défaut, unicité, format de valeur.
- À la fin de cette leçon, je sais modéliser une situation simple du quotidien (école, boutique, hôpital) sous forme de tables reliées.

PRÉREQUIS

- Notion de fichier et de dossier (organisation des données sur un ordinateur)

- Notion de logiciel et de système d'exploitation (leçons du Module 1)
- Types de données élémentaires : texte, nombre, date (tableur, classe de Seconde)
- Notion de tableau (lignes et colonnes) manipulée avec un tableur

1 DONNÉES, BASES DE DONNÉES ET SGBD

Situation d'accroche. Septembre, rentrée des classes au lycée de Nkongsamba. Au secrétariat, trois employés s'affairent autour d'une pile de cahiers : un cahier pour les inscriptions, un autre pour les paiements de la scolarité, un troisième pour les bulletins. Les 1 200 élèves de l'établissement y figurent, recopiés à la main, ligne après ligne. Chaque année, le même rituel : trois semaines entières pour recopier les noms, les dates de naissance, les classes, les montants versés. Inévitablement, des erreurs se glissent : « Ngo Bell Épouse » inscrite deux fois avec deux dates de naissance différentes, un élève de Terminale D encore enregistré en Première, un paiement de 25 000 FCFA noté dans le cahier des inscriptions mais oublié dans celui de la comptabilité. Quand un parent vient réclamer un duplicata de bulletin, il faut fouiller vingt cahiers pendant une heure.

Le proviseur, de retour d'un séminaire à Douala, entend parler d'un « logiciel de gestion scolaire » qui stocke toutes ces informations dans une base de données. En quelques secondes, on retrouve un élève, on imprime son bulletin, on vérifie ses paiements. Comment organiser ces informations pour les retrouver, les modifier et les protéger efficacement ? C'est toute la problématique de cette leçon. Pour y répondre, nous devons d'abord comprendre trois notions fondamentales : la **donnée**, la **base de données** et le **SGBD**.

1.1 De la donnée à l'information

Qu'est-ce qu'une donnée ?

Observe la fiche d'inscription d'un élève : « Mbarga Jean », « 15/03/2008 », « Première C », « 25 000 ». Pris isolément, chacun de ces éléments est un simple fait brut, sans traitement ni interprétation. C'est exactement ce qu'on appelle une donnée.

DÉFINITION Donnée

Une **donnée** est un élément brut, élémentaire, qui décrit un fait, un objet, une personne ou un événement du monde réel. Elle peut prendre la forme d'un texte (un nom), d'un nombre (un âge, un montant), d'une date, d'une image, d'un son ou d'une vidéo.

Une donnée possède généralement trois caractéristiques :

- un **nom** (ou identifiant) qui indique ce qu'elle représente : `nom_eleve`, `montant_paye` ;
- un **type** qui précise sa nature : texte, nombre entier, nombre décimal, date, etc. ;
- une **valeur** : « Mbarga Jean », 25 000, 15/03/2008.

Donnée ou information ? La distinction est subtile mais essentielle. La liste brute des notes « 12 ; 8 ; 15 ; 11 » est un ensemble de données. Une fois traitées (calcul de la moyenne : $\frac{12 + 8 + 15 + 11}{4} = 11,5$), elles deviennent une **information** utile : « l'élève a une moyenne de 11,5/20, il est passable ». L'information est donc une donnée **interprétée, organisée ou traitée**, qui prend du sens dans un contexte et permet de prendre une décision.

À RETENIR Donnée vs information

La **donnée** est la matière première brute ; l'**information** est le produit fini obtenu après traitement (calcul, tri, regroupement, comparaison). Le rôle d'un système informatique est précisément de transformer les données en informations.

1.2 Les limites du stockage dans des fichiers ou des cahiers

Revenons à notre secrétariat de Nkongsamba. Pourquoi la gestion sur cahiers (ou même dans de simples fichiers texte indépendants sur un ordinateur) pose-t-elle tant de problèmes ? Analysons la situation de manière rigoureuse.

- **Redondance des données.** Le nom et la date de naissance d'un élève sont recopiés dans le cahier des inscriptions, celui des paiements, celui des bulletins, celui de la cantine. La même donnée est dupliquée quatre fois : gaspillage d'espace et de temps.
- **Incohérence.** Conséquence directe de la redondance : si l'élève corrige sa date de naissance, il faut penser à la modifier partout. En pratique, on l'oublie dans un cahier sur deux, et l'établissement se retrouve avec deux valeurs contradictoires pour la même donnée.
- **Difficulté de recherche.** Retrouver « tous les élèves de Première D qui n'ont pas encore payé la deuxième tranche » exige de lire des centaines de lignes à la main et de croiser plusieurs cahiers. Avec un fichier texte classique, c'est à peine meilleur : aucun mécanisme de recherche croisée n'existe.
- **Absence de sécurité.** Un cahier peut être perdu, brûlé, mouillé, ou consulté et modifié par n'importe qui. Aucun contrôle d'accès, aucune sauvegarde automatique.
- **Difficulté de partage.** Deux employés ne peuvent pas consulter le même cahier en même temps. Les mises à jour simultanées sont impossibles.
- **Pas de contrôle de validité.** Rien n'empêche d'écrire une date de naissance « 32/15/2010 » ou un montant négatif : le cahier accepte tout.

EXEMPLE Comparaison : gestion papier ou base de données ?

Le tableau ci-dessous résume le contraste entre les deux approches pour le lycée de Nkongsamba.

Critère	Gestion sur cahiers	Gestion par base de données
Saisie des élèves	Recopie manuelle dans 4 cahiers (3 semaines)	Saisie unique, réutilisée partout (2 jours)
Recherche d'un élève	Feuilletage long, erreurs possibles	Quelques secondes par matricule ou nom
Cohérence	Doublons et contradictions fréquents	Donnée stockée une seule fois, cohérente
Sécurité	Aucune (perte, vol, détérioration)	Mots de passe, droits d'accès, sauvegardes
Accès simultané	Impossible	Plusieurs utilisateurs en même temps
Contrôle des valeurs	Aucun	Contraintes automatiques (format, unicité)

C'est précisément pour résoudre tous ces problèmes qu'ont été inventées les bases de données.

1.3 La notion de base de données

DÉFINITION Base de données

Une **base de données** (BD) est un ensemble structuré et organisé de données liées entre elles, stockées sur un support informatique, représentant un aspect du monde réel (un lycée, une banque, un hôpital) et accessibles à plusieurs utilisateurs de manière cohérente et durable.

Décomposons cette définition pour bien la comprendre :

- « structuré et organisé » : les données ne sont pas jetées en vrac ; elles sont rangées dans des **tables** (tableaux à lignes et colonnes) selon des règles précises, que nous étudierons dans la section 3 ;
- « de données liées entre elles » : les élèves sont liés à leurs classes, à leurs paiements, à leurs notes ; ces liens évitent la redondance ;
- « accessibles à plusieurs utilisateurs » : le secrétaire, le comptable et le censeur peuvent travailler simultanément sur les mêmes données, chacun avec ses droits.

EXEMPLE La base de données des élèves du lycée de Nkongsamba

La base de données du lycée pourrait contenir une table `ELEVE` dont voici un extrait :

Matricule	Nom et prénom	Classe	Date de naissance
NK2024-001	Mbarga Jean	1ère C	15/03/2008
NK2024-002	Ngo Bell Épouse	1ère D	02/07/2008
NK2024-003	Fotso Ulrich	1ère C	21/11/2007
NK2024-004	Kamdem Larissa	1ère E	09/01/2008

Chaque ligne décrit un élève, chaque colonne décrit une caractéristique. Une autre table **PAIEMENT** enregistrerait les versements, reliée aux élèves par leur matricule : ainsi, le nom de l'élève n'est jamais recopié dans la table des paiements, ce qui supprime la redondance.

ATTENTION Base de données ≠ simple fichier

Une base de données n'est pas un simple fichier texte ou un classeur de tableur. C'est un ensemble de données structurées et liées, gérées par un logiciel spécialisé qui garantit la cohérence, la sécurité et le partage. Un tableur peut certes contenir des tableaux, mais il ne gère ni les liens entre tables, ni les accès simultanés, ni les contraintes d'intégrité de manière rigoureuse.

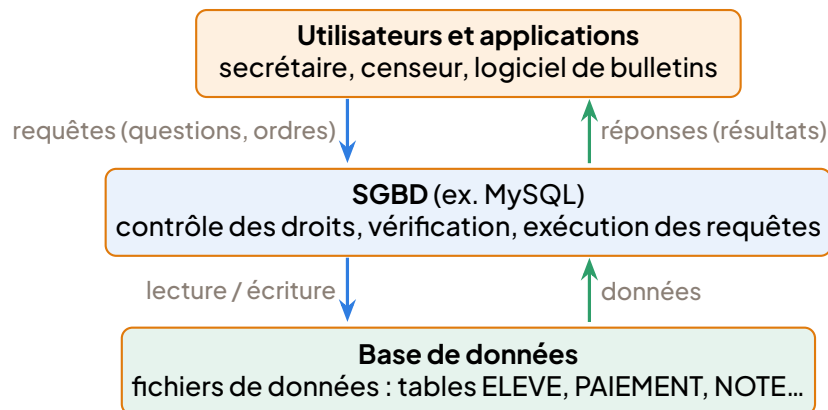
1.4 Le SGBD : le logiciel qui gère la base de données

Une base de données contient parfois des millions de données réparties dans des dizaines de tables. Il est hors de question de manipuler directement les fichiers sur le disque dur : ce serait dangereux et inefficace. On passe donc par un logiciel intermédiaire spécialisé.

DÉFINITION SGBD

Un **SGBD** (Système de Gestion de Bases de Données) est un logiciel qui permet de créer, définir, manipuler (interroger, insérer, modifier, supprimer) et administrer (sécuriser, sauvegarder) une base de données. Il joue le rôle d'intermédiaire obligé entre les utilisateurs et les données.

Le SGBD fonctionne comme un guichetier rigoureux : l'utilisateur ne touche jamais les données directement. Il formule une **requête** (une demande, par exemple « donne-moi la liste des élèves de Première C »), le SGBD vérifie ses droits, contrôle la validité de la demande, exécute la recherche dans les fichiers, puis renvoie le **résultat**. Cette architecture en couches est fondamentale.



Architecture en couches : le SGBD est l'intermédiaire obligatoire entre l'utilisateur et les données.

PROPRIÉTÉ Services rendus par un SGBD

Tout SGBD digne de ce nom assure au minimum les services suivants :

1. la définition des données : création de la base et de ses tables, choix des types de champs ;
2. la manipulation des données : insertion, consultation, mise à jour, suppression ;
3. le contrôle d'intégrité : rejet des valeurs invalides ou incohérentes ;
4. la sécurité : comptes utilisateurs, mots de passe, droits d'accès différenciés ;
5. le partage : accès simultané de plusieurs utilisateurs sans conflit ;
6. la reprise sur panne : sauvegardes et restauration des données.

ATTENTION Ne pas confondre base de données et SGBD

C'est l'erreur classique du Probatoire ! La base de données est le contenu : l'ensemble des données organisées (les élèves, les paiements, les notes). Le SGBD est le contenant logiciel : le programme qui crée, gère et protège ce contenu. Analogie : la base de données est l'argent déposé à la banque ; le SGBD est la banque elle-même, avec ses guichets, ses coffres et ses règles. On dit « j'interroge la base de données à travers le SGBD ».

1.5 Exemples de SGBD et domaines d'application

Il existe de nombreux SGBD, adaptés à des besoins variés. En voici les plus répandus :

SGBD	Caractéristiques	Usage typique
MySQL	Libre et gratuit, rapide, très répandu	Sites web, applications de gestion
PostgreSQL	Libre, très puissant et rigoureux	Grandes administrations, entreprises
Oracle	Commercial, très robuste, coûteux	Banques, grandes entreprises
Microsoft Access	Commercial, simple, intégré à Microsoft Office	Petites bases de données bureautiques
SQLite	Libre, ultra-léger, embarqué dans un seul fichier	Téléphones, applications mobiles

Les bases de données sont aujourd'hui omniprésentes dans la vie quotidienne camerounaise et mondiale :

- **Éducation** : gestion des élèves, des notes et des bulletins dans les lycées et universités ;
- **Banques** : Afriland First Bank, BICEC et les autres banques gèrent les comptes, les soldes et les transactions de leurs clients dans des bases de données sécurisées ;
- **Santé** : dossiers médicaux des patients, stocks de médicaments dans les hôpitaux ;
- **Téléphonie mobile** : MTN et Orange Cameroun enregistrent les abonnés, les forfaits, l'historique des appels et les transactions Mobile Money ;
- **Administration** : impôts (la DGI gère les contribuables), état civil (naissances, mariages, décès), élections ;
- **Commerce** : stocks et ventes des boutiques, supermarchés et stations-service.

LESAVAIS-TU ? Des bases de données nationales au Cameroun

Ta carte nationale d'identité et ta carte SIM sont enregistrées dans de gigantesques bases de données nationales ! Lors de l'identification des abonnés téléphoniques, chaque carte SIM vendue au Cameroun est associée, dans les bases de données des opérateurs, au numéro de CNI de son propriétaire. De même, le système de production des CNI s'appuie sur une base de données centralisée contenant les informations d'état civil de millions de Camerounais. Ces bases contiennent des dizaines de millions d'enregistrements et fonctionnent 24 heures sur 24 : impossible à imaginer avec des cahiers !

1.6 Exercice résolu et synthèse

EXERCICE RÉSOLU 1 Identifier données, BD et SGBD

Énoncé. Le lycée de Nkongsamba a finalement acquis le logiciel « ScolGest », construit autour du SGBD MySQL. Le secrétaire y saisit : « Kamdem Larissa », « 09/01/2008 », «

1ère E », « 25 000 FCFA versés le 04/10/2024 ».

1. Citer quatre données présentes dans cet énoncé.
2. Donner un exemple d'information que le proviseur peut obtenir à partir de ces données.
3. Dans cette situation, qu'est-ce qui constitue la base de données? Qu'est-ce qui constitue le SGBD?
4. Citer deux problèmes de la gestion sur cahiers que cette solution élimine.

SOLUTION

Solution détaillée.

1. Quatre données : le nom « Kamdem Larissa » (texte), la date de naissance « 09/01/2008 » (date), la classe « 1ère E » (texte), le montant « 25 000 FCFA » (nombre).
2. Information possible : après traitement (somme des versements et comparaison aux frais exigés), le proviseur apprend que « Kamdem Larissa a soldé sa première tranche de scolarité ». C'est une donnée traitée devenue utile à la décision.
3. La base de données est l'ensemble structuré des données du lycée (tables des élèves, des paiements, des notes) stockées sur l'ordinateur. Le SGBD est MySQL, le logiciel qui crée, manipule et protège ces données ; « ScolGest » est l'application qui dialogue avec MySQL au nom du secrétaire.
4. La redondance (le nom de l'élève n'est saisi qu'une seule fois) et la difficulté de recherche (on retrouve un élève en quelques secondes par son matricule). On peut aussi citer l'incohérence et l'absence de sécurité.

EXERCICE 1 À toi de jouer

La microfinance « Espoir de Bafoussam » gère les comptes de ses 800 clients dans des classeurs papier. Elle hésite à acheter un ordinateur équipé de Microsoft Access.

1. Définir les termes donnée, base de données et SGBD.
2. Citer trois limites de la gestion actuelle de la microfinance.
3. Expliquer, à l'aide d'un schéma en couches, le rôle d'intermédiaire joué par Access entre l'employé et les données des clients.
4. Proposer quatre champs (colonnes) pertinents pour une table `CLIENT` de cette microfinance.

CORRIGÉ Exercice 1

Solution.

1. Donnée : élément brut décrivant un fait (ex. : « 50 000 »). Base de données : ensemble structuré et organisé de données liées (ex. : l'ensemble des clients, comptes, opérations de la microfinance). SGBD : logiciel qui crée, manipule et administre la base de données (ex. : Microsoft Access).
2. Trois limites : redondance des informations (nom du client recopié sur plusieurs registres), incohérence (solde différent selon les registres), recherche lente (retrouver un client par son nom), absence de sécurité (vol, incendie, consultation non autorisée), pas d'accès simultané, pas de contrôle de validité (montant négatif possible).
3. Schéma en trois couches :
 - Couche haute : Utilisateur (employé de la microfinance) / Application (Microsoft Access).
 - Couche intermédiaire : SGBD (moteur Access/Jet/ACE) : vérification des droits, analyse de la requête, exécution.
 - Couche basse : Base de données (fichier .accdb) : tables CLIENT, COMPTE, OPERATION.

L'employé ne touche jamais au fichier .accdb directement ; il passe par l'interface Access (formulaires, requêtes) qui dialogue avec le moteur SGBD.
4. Quatre champs pour CLIENT :
 - id_client (entier, clé primaire) : identifiant unique.
 - nom_complet (texte) : nom et prénoms.
 - date_naissance (date) : pour vérifier la majorité.
 - telephone (texte) : contact, utile pour Mobile Money.
 - (Autres possibles : adresse, num_cni, date_inscription).

À RETENIR L'essentiel de la section

- Une **donnée** est un élément brut décrivant un fait ; traitée, elle devient une **information** utile à la décision.
- Le stockage dans des cahiers ou de simples fichiers entraîne redondance, incohérence, recherches lentes, absence de sécurité et de partage.
- Une **base de données** est un ensemble structuré et organisé de données liées entre elles.

- Un **SGBD** est le logiciel intermédiaire qui crée, manipule et administre la base de données : l'utilisateur ne touche jamais les données directement.
- Exemples de SGBD : MySQL, PostgreSQL, Oracle, Microsoft Access, SQLite. Les bases de données irriguent l'école, la banque, la santé, la téléphonie et l'administration camerounaise.

Maintenant que tu sais distinguer donnée, base de données et SGBD, la section suivante te fera découvrir les composants d'un SGBD : les « pièces » internes qui lui permettent d'accomplir toutes ces missions.

2 COMPOSANTS D'UN SGBD

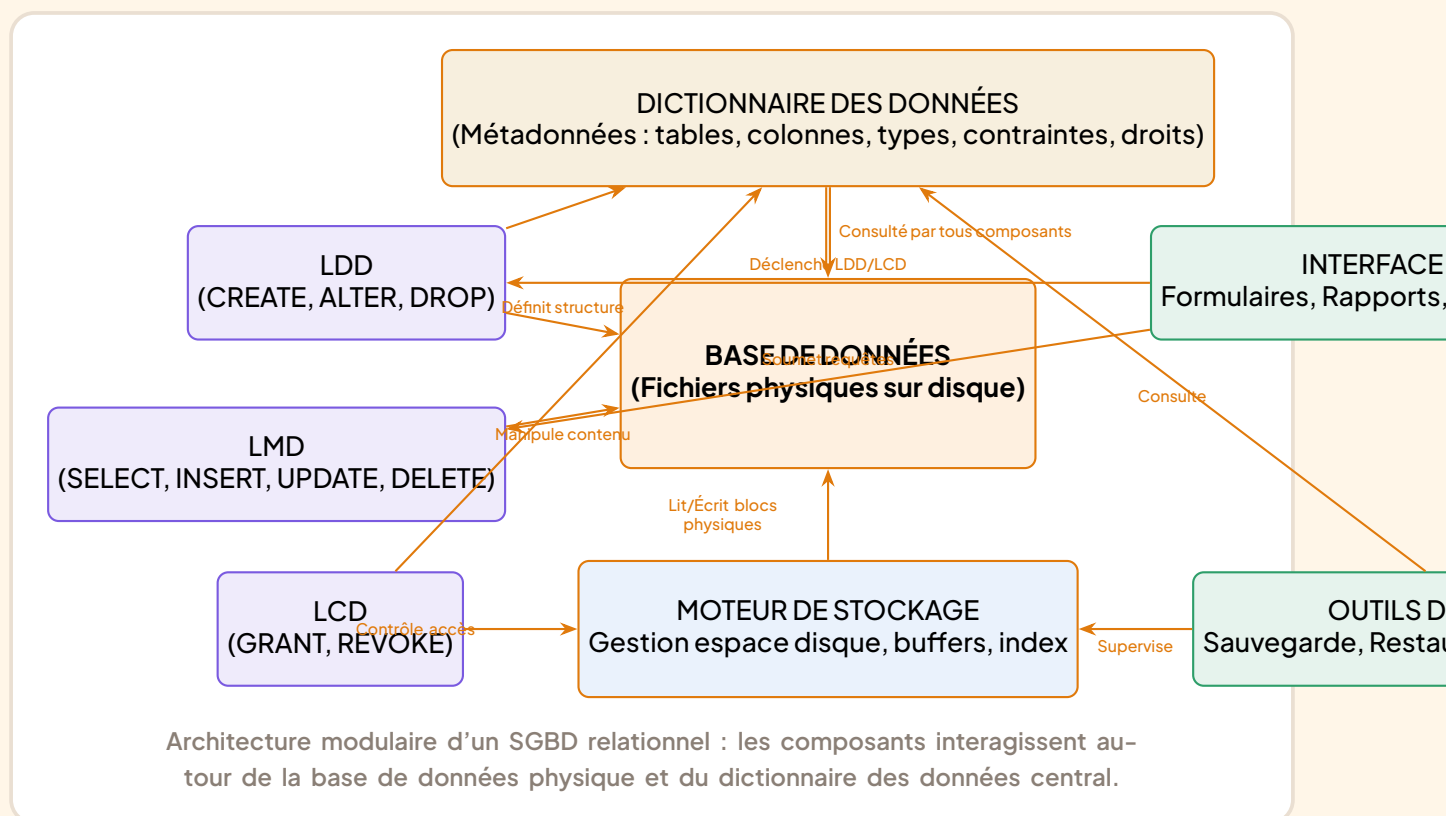
Dans la section précédente, nous avons distingué les **données** (faits bruts), la **base de données** (collection organisée et persistante) et le **SGBD** (logiciel pilote). Nous savons maintenant quoi stocker et qui gère le stockage. Il reste à comprendre comment ce logiciel s'organise intérieurement pour remplir sa mission. Un SGBD n'est pas un bloc monolithique : c'est une architecture modulaire où chaque composant a une responsabilité précise. C'est ce que nous allons disséquer ici, comme un mécanicien ouvre le capot d'un moteur pour en expliquer les organes.

2.1 Architecture générale : les cinq piliers du SGBD

Imaginez un grand dépôt de livres (la base de données) géré par une équipe de bibliothécaires (le SGBD). Pour que tout fonctionne, il faut :

- un **moteur de stockage** qui range physiquement les livres sur les étagères (disque dur) ;
- un **langage de définition (LDD)** pour construire de nouvelles étagères ou modifier l'agencement du dépôt ;
- un **langage de manipulation (LMD)** pour emprunter, rendre, ajouter ou retirer des livres ;
- un **langage de contrôle (LCD)** pour décider qui a le droit d'entrer dans quelle salle ;
- une **interface utilisateur** (guichets, catalogues numériques) pour que les lecteurs n'aient pas à parler le langage interne des bibliothécaires.

À cela s'ajoutent deux services transverses indispensables : le **dictionnaire des données** (le catalogue maître qui décrit la structure de chaque rayon) et les **outils d'administration** (inventaires, sauvegardes, restauration après sinistre).



2.2 Le moteur de stockage : le « cœur » invisible

DÉFINITION Moteur de stockage (Storage Engine)

C'est le composant bas niveau qui gère l'organisation physique des données sur le disque dur (ou SSD) : allocation des pages, gestion de la mémoire tampon (buffer pool), écriture/lecture des blocs, maintenance des index (B-arbres, tables de hachage) et gestion de la concurrence (verrous). C'est le seul composant qui « touche » réellement aux fichiers binaires.

EXEMPLE Exemple concret : MySQL et ses moteurs

MySQL (très utilisé dans les cybercafés et PME camerounaises pour sa gratuité) permet de choisir son moteur par table :

- InnoDB (par défaut depuis 2010) : supporte les transactions (ACID), les clés étrangères, verrouillage ligne par ligne. Idéal pour une gestion de stock ou de notes.
- MyISAM (ancien) : verrouillage table entière, pas de transactions, mais compression possible. Utile pour des tables en lecture seule (ex. : catalogue de produits figé).

Le choix du moteur impacte directement les performances et la fiabilité.

2.3 Les trois langages standards : LDD, LMD, LCD

Le standard SQL (Structured Query Language) regroupe en réalité trois sous-langages. Les connaître permet de savoir quelle commande taper selon l'objectif.

PROPRIÉTÉ Langage de Définition des Données — LDD (DDL en anglais)

Permet de créer, modifier et détruire la structure de la base (schéma). Les commandes principales sont :

- CREATE DATABASE / TABLE / INDEX / VIEW
- ALTER TABLE (ajouter/supprimer colonne, changer type, renommer)
- DROP DATABASE / TABLE / INDEX
- TRUNCATE TABLE (vider une table en gardant sa structure)

Caractéristique clé : ces commandes sont auto-validées (commit implicite) dans la plupart des SGBD ; on ne peut pas les annuler avec ROLLBACK.

PROPRIÉTÉ Langage de Manipulation des Données — LMD (DML en anglais)

Permet de traiter le contenu des tables (enregistrements). Commandes principales :

- SELECT : consultation (lecture seule)
- INSERT : ajout d'enregistrements
- UPDATE : modification d'enregistrements existants
- DELETE : suppression d'enregistrements

Ces opérations s'exécutent dans une transaction : on peut les annuler (ROLLBACK) ou les valider (COMMIT).

PROPRIÉTÉ Langage de Contrôle des Données — LCD (DCL en anglais)

Gère les droits d'accès (sécurité) :

- GRANT : accorder des privilèges (SELECT, INSERT, UPDATE, DELETE, EXECUTE, ALL) sur un objet à un utilisateur ou rôle.
- REVOKE : retirer des privilèges.

Exemple : GRANT SELECT, INSERT ON eleves TO secretaire_lycee;.

MÉTHODE Identifier le langage sollicité par une tâche

Pour chaque action demandée, détermine le sous-langage SQL concerné :

1. Action sur la structure ? (créer table, ajouter colonne, changer type, supprimer table) → LDD.
2. Action sur le contenu ? (ajouter un élève, modifier une note, lister les admis, supprimer un ancien) → LMD.
3. Action sur les droits ? (autoriser le prof à voir les notes, interdire la suppression au surveillant) → LCD.

EXERCICE RÉSOLU 2 Classement de commandes

Énoncé : Classez les commandes suivantes en LDD, LMD ou LCD :

1. `CREATE TABLE Professeur (...);`
2. `UPDATE Eleve SET moyenne = 14.5 WHERE matricule = 'E123';`
3. `GRANT SELECT ON Bulletin TO 'parent'@'localhost';`
4. `ALTER TABLE Eleve ADD COLUMN email VARCHAR(100);`
5. `DELETE FROM Absence WHERE date < '2023-09-01';`
6. `REVOKE INSERT ON Eleve FROM 'secretaire'@'%';`

SOLUTION

Solution détaillée :

1. LDD : création de structure (table).
2. LMD : modification de contenu (mise à jour d'un enregistrement).
3. LCD : gestion de droit (octroi de lecture).
4. LDD : modification de structure (ajout colonne).
5. LMD : suppression de contenu (enregistrements).
6. LCD : gestion de droit (retrait de privilège).

2.4 L'interface utilisateur : le pont vers l'humain

Le SGBD « nu » ne parle que SQL en ligne de commande. Pour un proviseur, un comptable ou un bibliothécaire, c'est inutilisable. L'**interface utilisateur** (IU) traduit les clics en requêtes SQL.

DÉFINITION Interface utilisateur d'un SGBD

Ensemble d'outils graphiques ou textuels permettant d'interagir avec la base sans écrire de SQL brut. On distingue :

- Formulaires : saisie/modification d'un enregistrement à la fois (ex. : fiche élève).
- États / Rapports : mise en page pour impression ou PDF (bulletins, listes de classe).
- Requêteurs graphiques (QBE — Query By Example) : construction de requêtes par glisser-déposer de tables/champs (Access, LibreOffice Base, pgAdmin).
- Tableurs de données : grille type Excel pour éditer directement le contenu d'une table.
- Outils d'administration graphiques : phpMyAdmin, pgAdmin, MySQL Workbench, SQL Server Management Studio.

EXPÉRIENCE TP guidé : Découvrir l'interface de phpMyAdmin (SGBD MySQL/MariaDB)

Objectif : Identifier visuellement les composants LDD, LMD, Interface et Dictionnaire dans un outil réel. Matériel : Navigateur web, serveur WAMP/XAMPP/LAMP local (ou hébergement gratuit type InfinityFree). Manipulation :

1. Connectez-vous à <http://localhost/phpmyadmin> (utilisateur `root`, mot de passe souvent vide en local).
2. Onglet « Bases de données » : cliquez « Créer une base » → nommez `lycee_bilingue`. Observation : c'est du LDD (`CREATE DATABASE`) via formulaire.
3. Sélectionnez la base, onglet « SQL » : tapez `CREATE TABLE Classe (id INT PRIMARY KEY, nom VARCHAR(30));`. Exécutez. Observation : zone de saisie SQL brute = Interface pour LDD.
4. Onglet « Structure » de la table `Classe` : vous voyez les colonnes, types, index. Observation : c'est la vue du Dictionnaire des données.
5. Onglet « Insérer » : formulaire pour ajouter une ligne (ex. `id=1, nom='1ere C'`). Observation : Interface générant du LMD (`INSERT`).
6. Onglet « Parcourir » : liste des enregistrements, boutons « Modifier », « Supprimer ». Observation : Interface pour LMD (`SELECT, UPDATE, DELETE`).
7. Onglet « Privilèges » : « Ajouter un compte utilisateur », cochez droits. Observation : Interface pour LCD (`GRANT`).

Interprétation : Un même outil (phpMyAdmin) expose tous les composants du SGBD. L'administrateur « voit » le dictionnaire, « construit » via LDD, « peuple » via LMD, « sécurise » via LCD.

2.5 Le dictionnaire des données : la « carte d'identité » de la base

DÉFINITION Dictionnaire des données (Data Dictionary / System Catalog)

Ensemble de tables système (souvent préfixées `sqlite_master`, `information_schema`, `sys`, `user_tables`) gérées automatiquement par le SGBD. Il stocke les métadonnées : nom des tables, colonnes, types de données, tailles, clés primaires/étrangères, contraintes, index, vues, procédures stockées, droits utilisateurs.

ATTENTION Piège fréquent : Confondre dictionnaire et données

Le dictionnaire décrit la structure (le « squelette »), pas le contenu (la « chair »).

- Dans le dictionnaire : « Table Eleve a une colonne `date_naissance` de type `DATE`, non nulle. »
- Dans les données : « Eleve n°123 a `date_naissance` = '2008-05-14'. »

Ne cherchez jamais une note d'élève dans `information_schema.columns` ! C'est une erreur de débutant qui coûte du temps en TP.

EXEMPLE Interrogation du dictionnaire (standard SQL `INFORMATION_SCHEMA`)

Pour lister toutes les tables de la base courante avec leur nombre de colonnes :

```
SELECT TABLE_NAME, COUNT(*) AS nb_colonnes
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = DATABASE() -- base courante (MySQL/MariaDB)
GROUP BY TABLE_NAME
ORDER BY TABLE_NAME;
```

Cette requête `SELECT` sur le dictionnaire est du LMD (on lit des tables système), mais elle renseigne sur la structure définie par le LDD.

2.6 Les outils d'administration : la boîte à outils du DBA

L'administrateur de base de données (DBA — Database Administrator) dispose d'utilitaires spécialisés, souvent en ligne de commande pour l'automatisation (scripts de nuit).

DÉFINITION Outils d'administration principaux

- Sauvegarde (Backup) : Export logique (`mysqldump`, `pg_dump` → fichier SQL) ou physique (copie fichiers bruts à l'arrêt ou via snapshot LVM/ZFS). Indispensable avant toute mi-

gration ou mise à jour.

- **Restauration (Recovery) :** Rejouer le fichier SQL (`mysql < backup.sql`) ou remise en place des fichiers physiques + relecture des journaux de transactions (WAL / Redo logs) pour récupérer l'état à la seconde près (Point-In-Time Recovery).
- **Monitoring / Supervision :** Vérifier l'espace disque, les requêtes lentes, les verrous, la réplication (ex. : `SHOW PROCESSLIST`, `pg_stat_activity`, Prometheus/Grafana).
- **Maintenance :** Reconstruction d'index (`ANALYZE`, `OPTIMIZE TABLE`), mise à jour des statistiques pour l'optimiseur de requêtes, purge des logs.
- **Gestion de la sécurité :** Rotation des mots de passe, audit des connexions, chiffrement (TLS pour le réseau, TDE — Transparent Data Encryption — pour les fichiers).

LESAVAIS-TU ?

SQL : un standard né dans les années 70, toujours roi

Le langage SQL a été conçu en 1974 par Donald Chamberlin et Raymond Boyce chez IBM (projet System R), sous le nom initial SEQUEL (Structured English Query Language). La première version commerciale est Oracle V2 (1979). La norme ANSI SQL-86 date de 1986, SQL-92 (1992) a posé les bases modernes (jointures `INNER/LEFT JOIN`), SQL:1999 a ajouté les types objets et déclencheurs, SQL:2003 le XML, SQL:2011 les données temporelles, SQL:2016 le JSON. 45 ans plus tard, tout SGBD relationnel (Oracle, SQL Server, PostgreSQL, MySQL, SQLite, MariaDB, DB2, Firebird) parle SQL. C'est l'un des rares standards informatiques à avoir survécu aussi longtemps sans être remplacé — preuve de sa puissance expressive pour l'algèbre relationnelle.

2.7 Synthèse : le flux d'une requête utilisateur

Pour fixer les idées, suivons le parcours d'une action simple : « Le proviseur clique sur « Imprimer bulletins 1ère C » dans l'application web de l'école. »

1. **Interface (Appli web) :** Le bouton déclenche un script côté serveur (Python/Node/Java/C#).
2. **Pilote (Driver JDBC/ODBC/PDO) :** Le script ouvre une connexion (pool) vers le SGBD, s'authentifie (LCD vérifie login/mot de passe/hôte).
3. **Analyseur / Optimiseur :** Le SGBD reçoit la requête `SELECT ... FROM Eleve JOIN Note ... WHERE classe='1ere C'`. Il consulte le **Dictionnaire** pour vérifier l'existence des tables/colonnes, les types, les index disponibles. Il choisit le meilleur plan d'exécution (utiliser l'index sur `classe`?).
4. **Moteur de stockage :** Exécute le plan : parcours d'index, lecture des pages en mémoire (buffer pool) ou sur disque, application des verrous partagés (lecture).
5. **Résultat :** Les lignes remontent vers le pilote → script → génération du PDF → téléchargement par le proviseur.

À RETENIR Bilan : Les 7 composants clés d'un SGBD

Composant	Rôle principal
Moteur de stockage	Écriture/lecture physique, index, concurrence, buffers
LDD (DDL)	Structure : CREATE, ALTER, DROP (auto-commit)
LMD (DML)	Contenu : SELECT, INSERT, UPDATE, DELETE (transactionnel)
LCD (DCL)	Sécurité : GRANT, REVOKE
Interface utilisateur	Formulaires, rapports, requêteurs graphiques, admin web
Dictionnaire des données	Métadonnées (structure only), consulté par tous les modules
Outils d'administration	Sauvegarde, restauration, monitoring, maintenance, chiffrement

EXERCICE 2 Entraînement — Composants SGBD

1. Un développeur écrit un script Python qui exécute `cursor.execute("ALTER TABLE Eleve ADD COLUMN telephone VARCHAR(15)")`. Quel composant du SGBD traite cette commande ? Justifiez.
2. La secrétaire saisit une nouvelle élève via un formulaire Access. La requête `INSERT` générée échoue avec l'erreur « Duplicate entry 'E456' for key 'PRIMARY' ». Quel composant a détecté l'erreur ? Quel composant a physiquement tenté l'écriture ?
3. Le DBA lance `mysqldump --single-transaction lycee > backup_2024.sql`. Citez deux composants sollicités et leur rôle précis ici.
4. Vrai ou Faux ? « Le dictionnaire des données contient la liste de tous les élèves de 1ère C. » Justifiez.
5. Un proviseur veut donner le droit de lire les moyennes aux parents d'élèves via l'interface phpMyAdmin. Quels composants interviennent, dans quel ordre ?

CORRIGÉ Exercice n°1 — Composants SGBD

1. LDD (Analyseur/Optimiseur + Moteur). `ALTER TABLE` est une commande de définition de structure. Le parseur la classe en LDD, l'optimiseur n'a pas grand chose à faire (pas de plan d'exécution complexe), le moteur modifie les fichiers de définition de

table (fichier `.frm` ou entrée dans `information_schema`) et potentiellement réorganise les pages de données si le type change la taille de ligne.

2. Détection : Moteur de stockage (via le gestionnaire de contraintes/index). L'index de clé primaire (B-arbre) refuse l'insertion car la valeur 'E456' existe déjà. Écriture physique tentée : Moteur de stockage (buffer pool → disque). L'interface Access a seulement transmis l'ordre.
3. 1. Interface d'administration (outil en ligne de commande `mysqldump`) : lit la structure (LDD via dictionnaire) et les données (LMD `SELECT`) pour produire le fichier SQL.
2. Moteur de stockage : assure la cohérence de la lecture (`--single-transaction` démarre une transaction en isolation `REPEATABLE READ` pour ne pas bloquer les écritures).
3. Dictionnaire : consulté pour obtenir la liste des tables, colonnes, types, contraintes à dumper.
4. Faux. Le dictionnaire ne contient que la structure : « Table `Eleve` existe, a une colonne `classe` de type `VARCHAR(20)` ». La liste des élèves (contenu) est dans les fichiers de données gérés par le moteur, accessibles via `SELECT * FROM Eleve WHERE classe='1ere C'` (LMD).
5. Ordre : (1) Interface (phpMyAdmin onglet Privilèges) → (2) LCD (analyse `GRANT SELECT ON Moyenne TO 'parent'@'%'`) → (3) Dictionnaire (mise à jour des tables de privilèges `mysql.columns_priv` ou `information_schema`) → (4) Moteur (écriture physique des nouveaux droits sur disque). Le proviseur ne voit que l'interface ; le SGBD enchaîne LCD + Dictionnaire + Moteur.

3 TABLES, CHAMPS, ENREGISTREMENTS ET CLÉS

Dans la section précédente, nous avons démonté le moteur d'un SGBD : moteur de stockage, interpréteur de requêtes, interfaces, dictionnaire des données. Mais tout ce mécanisme n'a de sens que par rapport à l'objet qu'il organise : la **table**. C'est elle que le dictionnaire des données décrit, elle que le moteur stocke, elle que les requêtes interrogent. Cette section est donc le cœur de la leçon : nous allons apprendre à regarder une table comme un chirurgien regarde un corps — en nommant chaque organe (champ, enregistrement) et en comprenant ce qui la rend fiable (les clés).

3.1 La table : une structure en lignes et colonnes

DÉFINITION Table

Une **table** (ou relation) est une structure de données organisée en lignes et en colonnes, qui décrit un ensemble d'objets de même nature (élèves, produits, factures, emprunts de

livres...). Chaque table porte un nom qui désigne l'ensemble d'objets qu'elle représente : ELEVE, CLASSE, PRODUIT.

L'intuition est immédiate : tu connais déjà les tableaux. Le registre de présence du proviseur, le relevé de notes trimestriel, le stock de la cantine — tous sont des tableaux où chaque ligne concerne un objet et chaque colonne une information. La table d'une base de données reprend exactement cette idée, mais avec une discipline plus stricte : **toutes les lignes décrivent le même type d'objet**, et **chaque colonne a un nom et un type de données fixés à l'avance**.

ATTENTION Ce qu'une table n'est pas

Contrairement à une feuille de tableur, une table ne tolère aucune fantaisie : pas de ligne « TOTAL » au milieu, pas de cellule contenant à la fois un texte et un nombre, pas de colonne sans nom. Cette rigidité est précisément ce qui permet au SGBD de garantir la fiabilité des données.

3.2 Champs et enregistrements

DÉFINITION Champ

Un **champ** (ou colonne, ou attribut) est une caractéristique d'un objet décrit par la table. Chaque champ possède :

- un nom unique dans la table : `matricule`, `nom`, `date_naiss` ;
- un type qui précise la nature des valeurs acceptées : texte (chaîne de caractères), nombre (entier ou décimal), date, booléen...

L'ensemble des champs constitue la structure de la table.

DÉFINITION Enregistrement

Un **enregistrement** (ou ligne, ou n-uplet) est une occurrence d'un objet de la table : c'est l'ensemble des valeurs prises par les champs pour un objet précis. La table `ELEVE` contient autant d'enregistrements qu'il y a d'élèves inscrits.

Retenons l'image suivante : la **structure** (les champs) est définie une fois pour toutes à la création de la table ; les **enregistrements** (les lignes) viennent s'ajouter, se modifier et se supprimer au fil de la vie de la base. On dit souvent : « les colonnes sont fixes, les lignes vivent ».

EXEMPLE La table ELEVE du lycée

Le secrétariat d'un lycée de Douala gère ses élèves dans une table `ELEVE` à quatre champs. Voici son contenu :

matricule	nom	date_naiss	quartier
E001	Ngo Bell	12/03/2008	Bonabéri
E002	Kamga Yves	25/07/2007	Deïdo
E003	Ngo Bell	30/11/2008	Akwa
E004	Mbappe Larissa	05/01/2008	Bali

Chaque colonne est un **champ** (`matricule`: texte; `nom`: texte; `date_naiss`: date; `quartier`: texte). Chaque ligne est un **enregistrement** : la ligne E002 décrit un élève précis, Kamga Yves.

clé primaire
(soulignée)



Table : ELEVE			
<u>matricule</u>	nom	date_naiss	quartier
E001	Ngo Bell	12-03-2008	Bonabéri
E002	Kamga Yves	25-07-2007	Deïdo
E003	Ngo Bell	30-11-2008	Akwa
E004	Mbappe Larissa	05-01-2008	Bali

→ champs
(colonnes)

→ enregistrements
(lignes)

Anatomie de la table ELEVE : les champs forment les colonnes, les enregistrements les lignes; le champ `matricule`, souligné, est la clé primaire.

3.3 La clé primaire : l'identité de chaque enregistrement

Posons un problème concret. Le proviseur demande au secrétariat : « Rayez **Ngo Bell** de la liste, il a déménagé. » Or la table contient **deux** élèves nommés Ngo Bell. Lequel faut-il supprimer? Impossible de répondre avec le seul nom : les deux enregistrements sont indistinguables par ce champ. Il faut un identifiant qui ne trompe jamais. C'est exactement le rôle de la **clé primaire**.

DÉFINITION Clé primaire

La **clé primaire** d'une table est un champ (ou un groupe de champs) qui identifie de façon unique chaque enregistrement de la table. Deux enregistrements ne peuvent jamais avoir la même valeur de clé primaire. Par convention, on souligne la clé primaire dans le schéma de la table.

PROPRIÉTÉ Propriétés de la clé primaire — admises

Toute clé primaire vérifie deux propriétés, garanties automatiquement par le SGBD :

1. **Unicité** : deux enregistrements distincts ont toujours des valeurs de clé primaire différentes ;
2. **Non-nullité** : la clé primaire ne peut jamais être vide (nulle).

Justification par l'exemple : si deux élèves portaient le même matricule E001, une recherche sur E001 retournerait deux réponses et l'on ne saurait plus lequel noter, lequel facturer, lequel radier : les deux enregistrements seraient indistinguables. De même, un enregistrement sans matricule serait un « fantôme » impossible à retrouver. La clé primaire est donc à l'enregistrement ce que le numéro de CNI est au citoyen.

ATTENTION Erreur fréquente : choisir le nom comme clé primaire

Beaucoup de débutants choisissent le champ `nom` comme clé primaire « parce que c'est parlant ». C'est une faute grave : les homonymes existent ! Dans notre table, deux élèves s'appellent Ngo Bell. Si `nom` était la clé primaire, le SGBD refuserait l'inscription du second Ngo Bell... ou pire, confondrait les deux. Une bonne clé primaire est une valeur stable, sans signification métier, et attribuée par le système : un matricule (E001), un numéro de série, un code produit. Règle d'or : on n'utilise jamais une donnée « naturelle » (nom, téléphone, date de naissance) comme clé primaire si l'on peut créer un identifiant artificiel.

EXERCICE RÉSOLU 3 Trouver la bonne clé primaire

Énoncé. Une boutique de Bonanjo veut créer une table `PRODUIT` avec les champs : `designation`, `prix`, `quantite`. Le gérant propose `designation` comme clé primaire. Est-ce un bon choix ? Proposer une meilleure solution.

SOLUTION

Solution détaillée.

1. Critère d'unicité : deux produits peuvent porter la même désignation (deux marques

de « Savon 100 g »). La désignation n'est donc pas garantie unique.

2. Critère de stabilité : le gérant peut renommer un produit (« Savon 100 g » devient « Savon 100g — nouveau format »); modifier une clé primaire en cours de vie de la base est dangereux.
3. Conclusion : `designation` est un mauvais choix. On ajoute un identifiant artificiel `code_prod` (P001, P002, ...) attribué par le système, unique et jamais nul. La structure devient : `PRODUIT(code_prod, designation, prix, quantite)`.

3.4 La clé étrangère : relier les tables entre elles

Une base de données réelle contient **plusieurs tables**. Le lycée gère les élèves, mais aussi les classes, les matières, les notes. Comment exprimer le fait que « l'élève E001 est en Première C2 » ?

Première idée (naïve) : recopier dans la table `ELEVE` toutes les informations de la classe (nom, salle, professeur principal...). Catastrophe : ces informations seraient **dupliquées** pour chacun des 60 élèves de la classe, et toute correction devrait être faite 60 fois, avec un risque d'incohérence énorme. C'est le problème de **redondance** que le modèle relationnel a été inventé pour éliminer.

Bonne idée : créer une table `CLASSE` à part, puis **référencer** la classe de chaque élève par sa clé primaire.

DÉFINITION Clé étrangère

Une **clé étrangère** est un champ d'une table qui fait référence à la clé primaire d'une autre table (ou, plus rarement, de la même table). Elle établit un lien entre les enregistrements des deux tables. Par convention, on la précède du symbole # dans le schéma relationnel.

MÉTHODE Relier deux tables : la règle « parent / enfant »

Pour relier deux tables liées par une relation du type « une classe contient plusieurs élèves » :

1. Identifier la table « parent » (celle du côté « un » : ici `CLASSE`, car une classe concerne plusieurs élèves) et la table « enfant » (côté « plusieurs » : ici `ELEVE`).
2. S'assurer que la table parent possède une clé primaire (ici `id_classe`).
3. Placer cette clé primaire comme clé étrangère dans la table enfant : on ajoute le champ `#id_classe` dans `ELEVE`.
4. Chaque enregistrement enfant reçoit, dans sa clé étrangère, la valeur de la clé primaire de son parent : l'élève E001 en classe C2 porte `#id_classe = C2`.

Ainsi, l'information « la classe C2 a pour professeur principal M. Etoundi » n'est écrite qu'une seule fois, dans la table CLASSE.

EXEMPLE Les tables ELEVE et CLASSE reliées

Table CLASSE (parent) :

id_classe	libelle	prof_principal
C2	Première C2	M. Etoundi
D1	Première D1	Mme Fouda

Table ELEVE (enfant) — la dernière colonne est la clé étrangère :

matricule	nom	date_naiss	#id_classe
E001	Ngo Bell	12/03/2008	C2
E002	Kamga Yves	25/07/2007	C2
E003	Ngo Bell	30/11/2008	D1
E004	Mbappe Larissa	05/01/2008	D1

Grâce à la clé étrangère, on sait que les deux Ngo Bell ne sont même pas dans la même classe : E001 est en C2, E003 en D1. Pour connaître le professeur principal de E003, on suit la flèche : #id_classe = D1 → ligne D1 de CLASSE → Mme Fouda.

CLASSE	
<u>id_classe</u>	libelle
C2	Première C2
D1	Première D1

ELEVE		
<u>matricule</u>	nom	#id_classe
E001	Ngo Bell	C2
E002	Kamga Yves	C2
E003	Ngo Bell	D1

la clé étrangère de ELEVE
référence la clé primaire de CLASSE

Lien entre les tables ELEVE et CLASSE : la clé étrangère #id_classe de la table enfant pointe vers la clé primaire id_classe de la table parent.

3.5 Le schéma relationnel : la carte d'identité de la base

Pour décrire la structure d'une base sans dessiner toutes les tables, les informaticiens utilisent une notation compacte : le **schéma relationnel**.

DÉFINITION Schéma relationnel

Le **schéma relationnel** d'une table s'écrit : NOM_TABLE suivi, entre parenthèses, de la liste de ses champs, avec la clé primaire soulignée et les clés étrangères précédées de #.

Pour notre lycée, le schéma relationnel complet s'écrit :

```
CLASSE(id_classe, libelle, prof_principal)
ELEVE(matricule, nom, date_naiss, #id_classe)
```

Dans le document imprimé que tu lis, les clés primaires apparaissent soulignées : CLASSE(id_classe, libelle, prof_principal) et ELEVE(matricule, nom, date_naiss, #id_classe). Ce schéma se lit en une phrase : « un élève possède un matricule, un nom, une date de naissance, et appartient à une classe identifiée par id_classe ». C'est la notation que l'on te demandera de produire au Probatoire pour décrire une base de données.

À RETENIR L'essentiel de la section

- Une **table** décrit un ensemble d'objets de même nature, en lignes et colonnes.
- Un **champ** (colonne) est une caractéristique nommée et typée ; un **enregistrement** (ligne) est une occurrence d'objet.
- La **clé primaire** identifie chaque enregistrement de façon unique et n'est jamais nulle ; on préfère un matricule à un nom (homonymes!).
- La **clé étrangère** d'une table enfant référence la clé primaire d'une table parent : c'est le mécanisme qui relie les tables et élimine la redondance.
- Le **schéma relationnel** résume tout cela : ELEVE(matricule, nom, date_naiss, #id_classe).

EXERCICE 3 À toi de jouer

La bibliothèque scolaire veut gérer ses prêts. On considère les tables LIVRE (code_livre, titre, auteur) et EMPRUNT (num_emprunt, date_emprunt, date_retour).

1. Propose une clé primaire pour chaque table en justifiant.
2. Un élève peut faire plusieurs emprunts ; chaque emprunt concerne un seul livre et un seul élève. Complète la table EMPRUNT avec les clés étrangères nécessaires.

3. Écris le schéma relationnel complet des trois tables.

CORRIGÉ Exercice : bibliothèque scolaire

1. LIVRE : clé primaire `code_livre` (identifiant artificiel unique ; le titre ne convient pas car deux livres peuvent porter le même titre). EMPRUNT : clé primaire `num_emprunt`.
2. On applique la règle parent/enfant : ELEVE et LIVRE sont parents de EMPRUNT. On ajoute donc dans EMPRUNT les champs `#matricule` (référence vers ELEVE) et `#code_livre` (référence vers LIVRE).
3. Schéma relationnel :

ELEVE(matricule, nom, date_naiss, #id_classe)

LIVRE(code_livre, titre, auteur)

EMPRUNT(num_emprunt, date_emprunt, date_retour, #matricule, #code_livre)

Nous savons désormais décrire précisément le contenu d'une base : tables, champs, enregistrements et clés. Mais comment ces tables naissent-elles concrètement ? Comment crée-t-on une base, une table, comment ajoute-t-on une colonne ou change-t-on un type ? C'est l'objet de la section suivante : les **fonctions de définition d'une base de données**.

4 FONCTIONS DE DÉFINITION D'UNE BASE DE DONNÉES

Dans la section précédente, nous avons appris à lire la structure d'une table : ses champs (colonnes), ses enregistrements (lignes), sa clé primaire et ses clés étrangères. Nous avons décortiqué la table ÉLÈVE du lycée comme si elle existait déjà. Mais une question demeure : **qui a créé cette table, et comment ?** Qui a décidé que la colonne `matricule` contiendrait du texte et que `date_naiss` contiendrait une date ?

C'est précisément l'objet de cette section. Avant de pouvoir saisir le moindre enregistrement, l'administrateur de la base doit d'abord construire le contenant : créer la base de données, puis créer les tables avec leurs champs et leurs types. Ces opérations, qui portent sur la structure et non sur le contenu, s'appellent les **fonctions de définition**.

DÉFINITION Fonctions de définition d'une base de données

Les **fonctions de définition** d'une base de données sont l'ensemble des opérations qui portent sur la structure de la base et de ses tables, et non sur les données elles-mêmes. Elles comprennent :

- la création d'une base de données ;

- la création d'une table (nom, champs, types de données, clé primaire);
- la suppression d'une base de données ou d'une table;
- la modification de la structure d'une table existante : ajout d'une colonne, renommage d'une colonne, changement du type d'une colonne.

Dans le langage SQL, ces ordres forment le langage de définition de données (LDD, en anglais DDL : Data Definition Language) : CREATE, DROP, ALTER.

ATTENTION Définition ou manipulation ? Ne pas confondre

Créer une table `ÉLÈVE` vide, c'est de la définition. Insérer dans cette table la ligne de l'élève Amina Ngo Bell, c'est de la manipulation (section suivante). Question classique au Probatoire : « L'ordre `CREATE TABLE` relève-t-il des fonctions de définition ou de manipulation ? » Réponse : définition, car il construit la structure.

4.1 Création d'une base de données

Intuition. Une base de données est comme un classeur vide que l'on achète avant d'y ranger des fiches. Avant de créer la moindre table, il faut d'abord créer ce classeur et lui donner un nom. Le proviseur du Lycée de Nkol-Eton qui informatise son établissement commencera par créer une base nommée, par exemple, `gestion_lycee`.

PROPRIÉTÉ Syntaxe de création d'une base de données

En SQL, la création d'une base de données s'écrit :

```
CREATE DATABASE [IF NOT EXISTS] nom_de_la_base;
```

Le nom doit être explicite, sans espaces ni accents, et ne pas entrer en conflit avec une base existante. La clause optionnelle `IF NOT EXISTS` (supportée par MySQL, PostgreSQL, SQLite) évite une erreur si la base existe déjà. Certains SGBD (comme Access) créent la base sous forme d'un seul fichier ; d'autres (MySQL, PostgreSQL) gèrent plusieurs bases simultanément sur un serveur.

EXEMPLE Création de la base du lycée

```
-- Creation de la base de donnees de l'etablissement  
CREATE DATABASE IF NOT EXISTS gestion_lycee;
```

Après exécution de cet ordre, la base existe mais elle est vide : elle ne contient encore

aucune table. C'est un contenant prêt à recevoir des tables.

4.2 Création d'une table : nom, champs et types de données

Intuition. Créer une table, c'est fabriquer la grille vierge d'un registre : on décide du titre de chaque colonne et de la nature de ce qu'on pourra y écrire, avant d'écrire la moindre ligne. C'est l'étape la plus importante de la définition, car une structure mal pensée au départ coûte très cher à corriger ensuite.

MÉTHODE Avant de créer une table : la démarche en 4 étapes

Avant d'écrire le moindre ordre `CREATE TABLE`, il faut :

1. Lister les champs : recenser toutes les informations à mémoriser sur l'objet étudié (pour un élève : matricule, nom, prénom, date de naissance, classe...). On peut s'aider du dictionnaire de données vu en classe de Seconde.
2. Choisir le type de chaque champ : texte, entier, décimal, date ou booléen, selon la nature de l'information (voir le tableau ci-dessous).
3. Désigner la clé primaire : choisir le champ (ou le groupe de champs) qui identifie de façon unique chaque enregistrement (par exemple le matricule).
4. Rédiger l'ordre de création : traduire ces choix en SQL avec `CREATE TABLE`.

4.3 Le choix du type de données adapté

Le **type de données** d'un champ indique au SGBD quelle sorte de valeur le champ peut recevoir. Ce choix n'est pas anodin : il conditionne l'espace mémoire occupé, les contrôles automatiques effectués et les opérations possibles (on peut additionner des nombres, pas des textes).

Type SQL courant	Signification	Exemple d'usage au lycée
<code>VARCHAR(n)</code> ou <code>TEXT</code>	Suite de caractères (lettres, chiffres, symboles), longueur variable	nom, prénom, adresse, matricule
<code>INTEGER</code> (ou <code>INT</code>)	Nombre entier (sans partie décimale)	effectif d'une classe, numéro d'ordre
<code>DECIMAL(p, s)</code> ou <code>REAL</code>	Nombre à virgule fixe (précision p , échelle s) ou flottant	moyenne (13,75), montant de la scolarité
<code>DATE</code>	Date calendaire (jour, mois, année)	date de naissance, date d'inscription
<code>BOOLEAN</code>	Vrai ou faux uniquement (<code>TRUE/FALSE</code>)	« a payé la scolarité : oui/non »

ATTENTION Pièges classiques dans le choix des types

- Ne stocke jamais un nombre en format texte si tu dois faire des calculs dessus : une moyenne déclarée en `TEXT` ne pourra pas être triée ni moyennée correctement (« 9 » serait classé après « 15 » dans un tri alphabétique!).
- Un numéro de téléphone se déclare en `TEXT` (ou `VARCHAR`), pas en entier : le zéro initial (6 90 00 00 00) serait perdu, et on ne fait jamais de calcul sur un numéro de téléphone.
- Une date saisie en texte libre (« 12 mars 2008 » ou « 03/12/08 ») devient inexploitable pour les tris et les calculs d'âge : toujours utiliser le type `DATE` (format ISO AAAA-MM-JJ).

PROPRIÉTÉ Syntaxe de création d'une table

```
CREATE TABLE [IF NOT EXISTS] nom_table (  
    champ1 TYPE1,  
    champ2 TYPE2,  
    ...,  
    PRIMARY KEY (champ_cle)  
);
```

Chaque champ est déclaré par son nom suivi de son type. La clause `PRIMARY KEY` (contrainte de table) désigne la clé primaire. Les champs sont séparés par des virgules et l'ordre se termine par un point-virgule.

EXEMPLE Création de la table `ÉLÈVE`, commentée ligne par ligne

```
-- Creation de la table ELEVE dans la base gestion_lycee  
CREATE TABLE IF NOT EXISTS ELEVE (  
    matricule VARCHAR(20), -- identifiant unique de l'eleve  
    nom VARCHAR(100), -- nom et prenom de l'eleve  
    date_naiss DATE, -- date de naissance (type date)  
    PRIMARY KEY (matricule) -- le matricule est la cle primaire  
);
```

Ligne par ligne : la première ligne lance la création et nomme la table ; les trois lignes suivantes déclarent chaque champ avec son type et une longueur maximale pour les textes ; l'avant-dernière ligne impose que deux élèves ne puissent jamais avoir le même matricule ; la dernière ligne ferme la liste des champs et valide l'ordre.

4.4 Modification de la structure d'une table : `ALTER TABLE`

Intuition. En cours d'année, le secrétariat du lycée se rend compte qu'il faut aussi contacter les

parents par téléphone : la table ÉLÈVE, créée avec trois colonnes, doit accueillir une quatrième colonne. Faut-il tout détruire et recommencer ? Non : l'ordre `ALTER TABLE` permet de faire évoluer la structure d'une table existante **sans perdre les données déjà saisies**.

PROPRIÉTÉ Les trois modifications de structure avec ALTER TABLE (SQL standard)

- **Ajouter une colonne** : `ALTER TABLE nom_table ADD COLUMN nom_colonne TYPE;`
- **Renommer une colonne** : `ALTER TABLE nom_table RENAME COLUMN ancien_nom TO nouveau_nom;`
- **Modifier le type d'une colonne** : `ALTER TABLE nom_table ALTER COLUMN nom_colonne TYPE nouveau_type;` (syntaxe PostgreSQL ; MySQL utilise `MODIFY COLUMN nom_colonne nouveau_type;` ; SQLite ne supporte pas directement cette opération).

EXEMPLE Faire évoluer la table ÉLÈVE

```
-- 1. Ajout d'une colonne telephone (texte, a cause du zero initial)
ALTER TABLE ELEVE ADD COLUMN telephone VARCHAR(15);

-- 2. Renommage : la colonne nom devient nom_complet
ALTER TABLE ELEVE RENAME COLUMN nom TO nom_complet;

-- 3. Changement de type (exemple PostgreSQL) : l'effectif passe de texte a entier
ALTER TABLE CLASSE ALTER COLUMN effectif TYPE INTEGER;
```

Table ÉLÈVE (version 1)

matricule (VARCHAR)
nom (VARCHAR)
date_naiss (DATE)

Table ÉLÈVE (version 2)

matricule (VARCHAR)
nom (VARCHAR)
date_naiss (DATE)
telephone (VARCHAR)

Évolution de la structure de la table ÉLÈVE : l'ordre `ALTER TABLE ... ADD COLUMN` ajoute la colonne « téléphone » sans détruire les colonnes ni les enregistrements existants.

ATTENTION Modifier le type d'une colonne : risque de perte de données

Changer le type d'une colonne déjà remplie est une opération délicate. Si une colonne texte contenant « 12 », « 15 » et « absent » est convertie en entier, les valeurs conver-

tibles (« 12 », « 15 ») passeront, mais « absent » sera perdu ou provoquera une erreur selon le SGBD. De même, passer d'un type décimal à un type entier tronque les parties décimales : une moyenne de 13,75 deviendrait 13. Règle d'or : sauvegarder la table avant toute modification de type, et vérifier que les données existantes sont compatibles avec le nouveau type.

4.5 Suppression d'une base de données et d'une table

Intuition. À la fin d'une expérimentation, ou lorsqu'une application est abandonnée, on peut vouloir supprimer une table ou une base entière. C'est l'équivalent numérique de la destruction d'un registre papier : une fois détruit, impossible de revenir en arrière.

PROPRIÉTÉ Syntaxe de suppression

```
DROP TABLE [IF EXISTS] nom_table; -- supprime une table et tout son contenu
```

```
DROP DATABASE [IF EXISTS] nom_base; -- supprime une base entière
```

Ces ordres détruisent à la fois la structure et toutes les données qu'elle contient. La suppression est irréversible : il n'existe pas de « corbeille » ni d'annulation (Ctrl+Z) dans un SGBD pour ces ordres. La clause `IF EXISTS` évite une erreur si l'objet n'existe pas.

ATTENTION Supprimer une table efface définitivement toutes ses données

L'ordre `DROP TABLE ELEVE;` ne se contente pas de vider la table : il fait disparaître la table et tous les enregistrements des élèves qu'elle contenait. Avant toute suppression, il faut toujours effectuer une sauvegarde (copie de la base sur un support externe ou dans le cloud) et vérifier que l'on s'adresse bien à la bonne table et à la bonne base. Une faute de frappe dans le nom peut détruire en une seconde des années de données d'un établissement.

EXERCICE RÉSOLU 4 Construire la table des enseignants du lycée

Le Lycée de Nkol-Eton veut créer une table `ENSEIGNANT` mémorisant : le matricule de l'enseignant (identifiant unique), son nom complet, sa matière enseignée, son numéro de téléphone et sa date d'embauche. Écrire l'ordre SQL de création de cette table, puis l'ordre permettant d'ajouter ensuite une colonne `email` de type texte.

SOLUTION

Étape 1 — Liste des champs et choix des types. Matricule : `VARCHAR(20)` (identifiant, pas

de calcul); nom complet : VARCHAR(100); matière : VARCHAR(50); téléphone : VARCHAR(15) (zéro initial à préserver); date d'embauche : DATE. Étape 2 — Clé primaire. Le matricule identifie chaque enseignant de façon unique : c'est la clé primaire. Étape 3 — Rédaction des ordres SQL.

```
-- Creation de la table ENSEIGNANT
CREATE TABLE IF NOT EXISTS ENSEIGNANT (
    matricule    VARCHAR(20),
    nom_complet  VARCHAR(100),
    matiere      VARCHAR(50),
    telephone    VARCHAR(15),
    date_embauche DATE,
    PRIMARY KEY (matricule)
);

-- Ajout ulterieur de la colonne email
ALTER TABLE ENSEIGNANT ADD COLUMN email VARCHAR(100);
```

L'ordre ALTER TABLE préserve tous les enseignants déjà enregistrés : la nouvelle colonne email sera simplement vide (NULL) pour eux, en attendant d'être renseignée.

EXPÉRIENCE**TP guidé : créer et faire évoluer une base avec un SGBD libre**

Objectif : mettre en œuvre les fonctions de définition sur un SGBD réel (par exemple SQLite via DB Browser for SQLite, ou MySQL/MariaDB sur les postes de la salle multimédia). Matériel : un poste de travail avec un SGBD installé. Manipulation :

1. Créer une base nommée `biblio_lycee`.
2. Créer la table `LIVRE` avec les champs `code` (texte, clé primaire), `titre` (texte), `auteur` (texte) et `annee` (entier).
3. Vérifier dans l'interface du SGBD que la table apparaît avec ses quatre colonnes.
4. Ajouter la colonne `disponible` de type booléen avec `ALTER TABLE LIVRE ADD COLUMN disponible BOOLEAN;`
5. Renommer la colonne `annee` en `annee_edition` avec `ALTER TABLE LIVRE RENAME COLUMN annee TO annee_edition;`
6. Supprimer la table avec `DROP TABLE LIVRE;`, puis constater qu'elle a disparu de la liste des tables.

Observations et interprétation : après l'étape 4, la table existe toujours et ses éventuelles données sont intactes : la définition modifie la structure sans toucher au contenu. Après l'étape 6, la table et son contenu ont définitivement disparu : la suppression est irréversible, d'où l'importance des sauvegardes.

À RETENIR L'essentiel des fonctions de définition

- Les fonctions de définition portent sur la structure de la base : création et suppression de la base et des tables, ajout, renommage et changement de type des colonnes.
- Ordres SQL correspondants : CREATE DATABASE, CREATE TABLE, ALTER TABLE (ADD COLUMN, RENAME COLUMN, modification de type), DROP TABLE, DROP DATABASE.
- Avant de créer une table : lister les champs, choisir le type adapté à chacun (texte, entier, décimal, date, booléen) et désigner la clé primaire.
- ALTER TABLE fait évoluer la structure sans perdre les données ; DROP détruit structure et données de façon irréversible : toujours sauvegarder avant.

EXERCICE 4 À toi de jouer

La bibliothèque du Lycée Classique de Maroua veut gérer ses emprunts.

1. Propose la liste des champs de la table EMPRUNT avec le type de données adapté à chacun, et désigne une clé primaire.
2. Rédige l'ordre CREATE TABLE correspondant (avec IF NOT EXISTS).
3. Un mois plus tard, on veut mémoriser en plus la date de retour effective du livre : écris l'ordre SQL nécessaire.
4. Explique pourquoi il serait dangereux de supprimer cette table en fin d'année sans précaution.

5 FONCTIONS DE MANIPULATION D'UNE BASE DE DONNÉES

Dans la section précédente, nous avons appris à définir une base de données : créer la base elle-même, créer ses tables, ajouter ou renommer des colonnes, modifier le type d'un champ. Autrement dit, nous avons construit les contenants : la structure, le squelette. Mais une base de données vide ne sert à rien ! Ce qui donne sa valeur à la base du lycée, ce sont les données : les noms des élèves, leurs classes, leurs montants de scolarité payés. Cette section est consacrée aux **fonctions de manipulation** : tout ce qu'on peut faire sur les données une fois la structure en place.

DÉFINITION Les quatre fonctions de manipulation

Les **fonctions de manipulation** d'une base de données sont les opérations qui agissent sur le contenu des tables (les enregistrements), sans jamais modifier leur structure. On en distingue quatre :

- l'**insertion** : ajouter un nouvel enregistrement dans une table ;
- la **mise à jour** : modifier la valeur d'un ou plusieurs champs d'enregistrements existants ;
- la **suppression** : retirer un ou plusieurs enregistrements d'une table ;
- la **consultation** : rechercher et afficher des données selon des critères, sans les modifier.

À RETENIR Manipulation ou définition ? La distinction fondamentale

C'est LA question piège du Probatoire. Retiens cette règle simple :

- Définition = agir sur la structure (créer une table, ajouter une colonne, changer un type, supprimer une table). On touche au plan de la maison.
- Manipulation = agir sur les données (ajouter un élève, corriger un montant, effacer un enregistrement, afficher une liste). On touche aux habitants de la maison, pas aux murs.

Créer la table `ELEVE` relève de la définition ; y inscrire l'élève Amina relève de la manipulation.

Pour toute la suite, nous travaillerons sur la table `ELEVE` du lycée, créée à la section précédente :

num_eleve	nom	classe	scolarite
E001	MBARGA Paul	1ère C	25 000
E002	NGO Bell Amina	1ère D	25 000
E003	FOTSO Ulrich	1ère C	25 000
E004	KAMGA Larissa	1ère E	30 000

5.1 L'insertion : ajouter un nouvel enregistrement

Intuition. À la rentrée, une nouvelle élève, DJOMO Sandrine, s'inscrit en 1ère C. Son dossier n'existe pas encore dans la base : il faut créer une nouvelle ligne dans la table `ELEVE`. C'est exactement le rôle de l'insertion.

DÉFINITION Insertion

L'**insertion** est la fonction de manipulation qui consiste à ajouter un nouvel enregistre-

ment (une nouvelle ligne) dans une table, en fournissant une valeur pour chacun de ses champs.

PROPRIÉTÉ Syntaxe générale de l'insertion en SQL

```
INSERT INTO nom_table (champ1, champ2, champ3)
VALUES (valeur1, valeur2, valeur3);
```

Les valeurs sont données dans le même ordre que les champs déclarés. Les valeurs textuelles s'écrivent entre apostrophes; les valeurs numériques s'écrivent sans apostrophes.

EXEMPLE Inscrire une nouvelle élève

```
-- Inscription de DJOMO Sandrine en 1ère C
INSERT INTO ELEVE (num_eleve, nom, classe, scolarite)
VALUES ('E005', 'DJOMO Sandrine', '1ère C', 25000);
```

Après exécution, la table `ELEVE` contient une cinquième ligne : E005, DJOMO Sandrine, 1ère C, 25 000.

ATTENTION Pièges fréquents à l'insertion

- **Ordre des valeurs** : `VALUES ('E005', 25000, 'DJOMO Sandrine', ...)` mélange les champs : le SGBD refuse (ou pire, enregistre des données incohérentes si les types coïncident).
- **Doublon de clé primaire** : insérer un enregistrement avec `num_eleve = 'E001'` est refusé, car la clé primaire doit être unique (contrainte d'intégrité, voir section suivante).
- **Apostrophes oubliées** : un texte sans apostrophes provoque une erreur de syntaxe.

5.2 La mise à jour : modifier des enregistrements existants

Intuition. Le conseil d'administration décide d'ajuster la scolarité de l'élève E003, qui bénéficie d'un régime différent : son montant passe de 25 000 à 27 500 FCFA. L'élève existe déjà dans la base ; on ne crée pas de nouvelle ligne, on modifie la valeur d'un champ d'une ligne existante. C'est la mise à jour.

DÉFINITION Mise à jour

La **mise à jour** est la fonction de manipulation qui consiste à modifier la valeur d'un ou plusieurs champs d'un ou plusieurs enregistrements existants, sans changer le nombre de lignes de la table.

PROPRIÉTÉ Syntaxe générale de la mise à jour en SQL

```
UPDATE nom_table  
SET champ = nouvelle_valeur  
WHERE condition;
```

La clause `WHERE` désigne les enregistrements à modifier. Sans `WHERE`, la modification s'applique à toutes les lignes de la table.

EXEMPLE Exemple chiffré : corriger la scolarité de l'élève E003

On veut faire passer le montant de scolarité de l'élève E003 (FOTSO Ulrich) de 25 000 à 27 500 FCFA :

```
-- Mise a jour du montant de scolarite de l'eleve E003  
UPDATE ELEVE  
SET scolarite = 27500  
WHERE num_eleve = 'E003';
```

Déroulement : le SGBD parcourt la table, repère la ligne dont `num_eleve` vaut 'E003' (une seule, car c'est la clé primaire), puis remplace 25 000 par 27 500 dans le champ `scolarite`. Avant : E003, FOTSO Ulrich, 1ère C, 25 000. Après : E003, FOTSO Ulrich, 1ère C, 27 500. Les quatre autres lignes sont inchangées.

On peut aussi modifier plusieurs champs en une seule opération, en séparant les affectations par des virgules :

```
-- FOTSO Ulrich redouble et passe en classe de 1ere D  
UPDATE ELEVE  
SET classe = '1ere D', scolarite = 25000  
WHERE num_eleve = 'E003';
```

5.3 La suppression : retirer des enregistrements

Intuition. En novembre, l'élève KAMGA Larissa (E004) est transférée dans un autre lycée de Yaoundé. Son dossier ne doit plus figurer parmi les élèves actifs : il faut retirer la ligne correspondante de la table. C'est la suppression.

DÉFINITION Suppression

La **suppression** est la fonction de manipulation qui consiste à retirer un ou plusieurs enregistrements (des lignes entières) d'une table. La structure de la table (ses colonnes) est conservée ; seules les données disparaissent.

PROPRIÉTÉ Syntaxe générale de la suppression en SQL

```
DELETE FROM nom_table  
WHERE condition ;
```

La clause `WHERE` désigne les enregistrements à supprimer. Sans `WHERE`, tous les enregistrements de la table sont supprimés.

EXEMPLE Retirer l'élève transférée

```
-- Suppression du dossier de l'élève E004 (transferee)  
DELETE FROM ELEVE  
WHERE num_eleve = 'E004' ;
```

Après exécution, la ligne E004, KAMGA Larissa, 1ère E, 30 000 n'existe plus dans la table. Les autres lignes sont intactes.

ATTENTION Un `DELETE` sans condition vide entièrement la table !

C'est l'erreur la plus redoutable en manipulation de données. Si tu tapes :

```
DELETE FROM ELEVE; -- DANGER : pas de WHERE !
```

le SGBD supprime tous les élèves de la table, sans demander confirmation. La table existe encore (sa structure est intacte), mais elle est vide. Même danger avec `UPDATE` sans `WHERE` : tous les élèves se retrouveraient avec la même scolarité ! Règle d'or : avant d'exécuter un `DELETE` ou un `UPDATE`, vérifie toujours la clause `WHERE`. Un bon réflexe de professionnel consiste à tester d'abord la condition avec un `SELECT` (voir ci-dessous) pour visualiser les lignes qui seront touchées.

5.4 La consultation : rechercher et afficher des données

Intuition. Le proviseur demande au secrétariat : « Donne-moi la liste des élèves de 1ère C ». Personne ne modifie la base : on veut simplement lire une partie des données, celles qui satisfont un critère. C'est la consultation, de loin l'opération la plus fréquente dans la vie d'une base

de données.

DÉFINITION Consultation

La **consultation** est la fonction de manipulation qui consiste à rechercher et afficher des données selon des critères de sélection, sans les modifier. Elle s'effectue au moyen de **requêtes** : des questions formulées dans le langage de la base de données.

PROPRIÉTÉ Syntaxe générale de la consultation en SQL

```
SELECT champ1, champ2  
FROM nom_table  
WHERE condition ;
```

- SELECT indique les champs (colonnes) à afficher ; l'étoile * signifie « tous les champs » ;
- FROM indique la table interrogée ;
- WHERE (facultatif) précise le **critère de sélection** : seules les lignes qui vérifient la condition sont affichées. Sans WHERE, toutes les lignes sont affichées.

Le rôle du **critère de sélection** est fondamental : c'est lui qui transforme une table brute en réponse précise à une question. « Affiche tous les élèves » et « affiche les élèves de 1ère C » sont deux consultations différentes sur la même table ; seul le critère change.

EXEMPLE Deux consultations sur la table ELEVE

```
-- Consultation 1 : liste complete des eleves (tous les champs)  
SELECT * FROM ELEVE ;  
  
-- Consultation 2 : noms et scholarites des eleves de 1ere C  
SELECT nom, scholarite  
FROM ELEVE  
WHERE classe = '1ere C' ;
```

La consultation 2 n'affiche que les lignes dont le champ `classe` vaut '1ere C' : MBARGA Paul (25 000), FOTSO Ulrich (27 500) et DJOMO Sandrine (25 000). Les élèves des autres classes sont filtrés par le critère.

LESAVAIS-TU ? Tu consultes des bases de données tous les jours

Quand tu composes le code pour vérifier ton solde Mobile Money (MTN MoMo ou Orange

Money), tu déclenches en réalité une requête de consultation dans une immense base de données : le système recherche la ligne correspondant à ton numéro de téléphone et t'affiche le champ « solde ». Quand tu effectues un transfert, c'est une mise à jour : ton solde diminue et celui du destinataire augmente. Et quand un nouvel abonné active son compte, c'est une insertion. Les quatre fonctions de cette leçon font tourner l'économie numérique camerounaise !

5.5 Vue d'ensemble : effet des quatre opérations sur la table ELEVE

La figure ci-dessous résume l'effet de chaque fonction de manipulation sur notre table exemple. Observe bien : seule la consultation ne modifie pas le contenu de la table.

1. Insertion (INSERT)

E001 • E002 • E003 • E004
E001 MBARGA, 1C, 25 000
E002 NGO B., 1D, 25 000
E003 FOTSO, 1C, 25 000
E004 KAMGA, 1E, 30 000

↓ + E005 DJOMO, 1C, 25 000

E001 MBARGA, 1C, 25 000
E002 NGO B., 1D, 25 000
E003 FOTSO, 1C, 25 000
E004 KAMGA, 1E, 30 000
E005 DJOMO, 1C, 25 000

2. Mise à jour (UPDATE)

E001	25 000
E002	25 000
E003	25 000
E004	30 000
E005	25 000

↓ E003 : 25 000 → 27 500

E001	25 000
E002	25 000
E003	27 500
E004	30 000
E005	25 000

3. Suppression (DELETE)

E001	MBARGA, 1C
E002	NGO B., 1D
E003	FOTSO, 1C
E004	KAMGA, 1E
E005	DJOMO, 1C

↓ - E004 (transferee)

E001	MBARGA, 1C
E002	NGO B., 1D
E003	FOTSO, 1C
E005	DJOMO, 1C

4. Consultation (SELECT)

E001	MBARGA, 1C
E002	NGO B., 1D
E003	FOTSO, 1C
E005	DJOMO, 1C

↓ filtre : classe = '1ere C'

MBARGA Paul, 25 000
FOTSO Ulrich, 27 500
DJOMO Sandrine, 25 000

resultat affiche, table inchangée

Effet des quatre fonctions de manipulation sur la table ELEVE : insertion d'une ligne, mise à jour d'une valeur, suppression d'une ligne, consultation filtrée par un critère.

5.6 La démarche générale avant toute manipulation

Que ce soit pour insérer, modifier, supprimer ou consulter, un professionnel ne tape jamais un ordre au hasard. Il suit une démarche systématique.

MÉTHODE Identifier la table, la condition et les champs avant d'exécuter

Avant d'exécuter toute manipulation, réponds par écrit à ces trois questions :

1. Quelle table ? Identifie la table concernée (ici : ELEVE). Une erreur de table peut corrompre d'autres données.
2. Quelle condition ? Détermine précisément les enregistrements visés, idéalement par la clé primaire (WHERE num_eleve = 'E003'). Pour un UPDATE ou un DELETE, teste d'abord la condition avec un SELECT ... WHERE : les lignes affichées sont exactement celles qui seront touchées.
3. Quels champs ? Liste les champs à renseigner (insertion), à modifier (mise à jour) ou à afficher (consultation), avec leurs nouvelles valeurs.

Ce n'est qu'après ces trois vérifications que tu rédiges et exécutes l'ordre SQL.

EXERCICE RÉSOLU 5 Choisir la bonne fonction de manipulation

Énoncé. Pour chacune des situations suivantes vécues par le secrétariat du lycée, indique la fonction de manipulation concernée (insertion, mise à jour, suppression ou consultation) et rédige l'ordre SQL correspondant sur la table ELEVE(num_eleve, nom, classe, scolarite).

1. Le censeur veut connaître le nombre d'élèves de 1ère D.
2. L'élève E002, NGO Bell Amina, a finalement payé 30 000 FCFA au lieu de 25 000 FCFA.
3. Un nouvel élève, ETOA Jean (E006), s'inscrit en 1ère E avec une scolarité de 30 000 FCFA.
4. L'élève E005 abandonne ses études et quitte définitivement le lycée.

SOLUTION

Solution détaillée.

1. On lit des données sans les modifier : c'est une consultation. Le critère de sélection porte sur la classe.

```
SELECT * FROM ELEVE  
WHERE classe = '1ere D';
```

2. L'élève existe déjà ; on change la valeur d'un champ : c'est une mise à jour.

```
UPDATE ELEVE  
SET scolarite = 30000  
WHERE num_eleve = 'E002';
```

3. On crée une nouvelle ligne dans la table : c'est une insertion.

```
INSERT INTO ELEVE (num_eleve, nom, classe, scolarite)
VALUES ('E006', 'ETOA Jean', '1ere E', 30000);
```

4. On retire une ligne de la table : c'est une suppression. La condition porte sur la clé primaire pour ne viser qu'un seul enregistrement.

```
DELETE FROM ELEVE
WHERE num_eleve = 'E005';
```

Remarque la méthode : pour chaque situation, nous avons d'abord identifié la table, puis la condition, puis les champs concernés, avant de rédiger l'ordre.

EXERCICE 5 À toi de jouer

Sur la table `ELEVE(num_eleve, nom, classe, scolarite)`, rédige les ordres SQL permettant de :

1. afficher les noms de tous les élèves dont la scolarité est supérieure à 25 000 FCFA ;
2. augmenter de 2 000 FCFA la scolarité de tous les élèves de 1ère E ;
3. supprimer tous les élèves de la classe de 1ère D ;
4. expliquer ce qui se passe si l'on exécute `UPDATE ELEVE SET classe = '1ere C';` sans clause `WHERE`.

CORRIGÉ Exercice

1. Consultation avec critère sur la scolarité :

```
SELECT nom FROM ELEVE
WHERE scolarite > 25000;
```

2. Mise à jour ciblée sur une classe :

```
UPDATE ELEVE
SET scolarite = scolarite + 2000
WHERE classe = '1ere E';
```

3. Suppression conditionnelle :

```
DELETE FROM ELEVE
WHERE classe = '1ere D';
```

4. Sans clause `WHERE`, la mise à jour s'applique à toutes les lignes : tous les élèves du lycée se retrouvent en 1ère C. C'est une erreur grave qui détruit l'information. D'où la règle d'or : toujours vérifier la condition avant d'exécuter un `UPDATE` ou un `DELETE`.

À RETENIR L'essentiel de la section

- Les quatre fonctions de manipulation agissent sur les données : insertion (`INSERT INTO ... VALUES`), mise à jour (`UPDATE ... SET ... WHERE`), suppression (`DELETE FROM ... WHERE`) et consultation (`SELECT ... FROM ... WHERE`).
- La définition agit sur la structure ; la manipulation agit sur le contenu. Ne jamais confondre.
- La clause `WHERE` est le cœur de la sélection : un `UPDATE` ou un `DELETE` sans condition agit sur toute la table.
- Avant toute exécution : identifier la table, la condition et les champs concernés.

6 CONTRAINTES D'INTÉGRITÉ

Dans la section précédente, nous avons appris à manipuler les données : insérer, consulter, modifier et supprimer des enregistrements. Mais une question essentielle demeure : qu'est-ce qui empêche un utilisateur d'insérer n'importe quoi ? Imagine le secrétariat du lycée : un élève pourrait être enregistré deux fois avec le même matricule, une note de 27/20 pourrait être saisie, ou un élève pourrait être affecté à une classe qui n'existe pas. Sans protection, la base de données deviendrait rapidement un chaos inutilisable. C'est précisément pour éviter cela que le modèle relationnel introduit les **contraintes d'intégrité**.

6.1 Définition et rôle des contraintes d'intégrité

DÉFINITION Contrainte d'intégrité

Une **contrainte d'intégrité** est une règle définie sur une table (ou sur un champ) qui garantit l'exactitude et la cohérence des données stockées dans la base. Le SGBD vérifie automatiquement ces règles à chaque insertion ou mise à jour : toute opération qui violerait une contrainte est rejetée et provoque un message d'erreur.

L'intuition est simple : une contrainte d'intégrité joue le rôle d'un vigile placé à l'entrée de la base de données. Avant d'accepter une nouvelle donnée, le SGBD vérifie qu'elle respecte toutes les règles déclarées. Si une seule règle est enfreinte, la donnée n'entre pas.

Au programme de Première, on distingue **six contraintes d'intégrité** :

1. la contrainte de **clé primaire** ;
2. la contrainte de **valeur autorisée** (ou de domaine) ;
3. la contrainte de **valeur nulle** ;
4. la contrainte de **valeur par défaut** ;
5. la contrainte d'**unicité** ;
6. la contrainte de **format de valeur**.

On y ajoute, en complément très utile pour le Probatoire et le Baccalauréat, la contrainte d'**intégrité référentielle**, qui concerne les liens entre tables.

PROPRIÉTÉ Vérification automatique par le SGBD (admise)

Le SGBD vérifie automatiquement toutes les contraintes déclarées sur une table à chaque insertion (INSERT) et à chaque mise à jour (UPDATE). Si l'opération viole au moins une contrainte :

- l'opération est rejetée (la donnée n'est pas enregistrée) ;
- le SGBD affiche un message d'erreur indiquant la contrainte violée ;
- le contenu de la table reste inchangé.

Cette vérification ne coûte aucun effort au programmeur : il suffit de déclarer les contraintes au moment de la création de la table.

6.2 La contrainte de clé primaire

Rappelons que la **clé primaire** identifie de manière unique chaque enregistrement d'une table. La contrainte de clé primaire impose donc **deux règles simultanées** :

À RETENIR Les deux règles de la clé primaire

- **Unicité** : deux enregistrements ne peuvent jamais avoir la même valeur de clé primaire ;
- **Non-nullité** : la clé primaire ne peut jamais être vide (valeur nulle interdite).

Il ne peut exister qu'une seule clé primaire par table.

EXEMPLE Insertion refusée : matricule déjà existant

La table `ELEVE` possède déjà un élève de matricule `E001`. On tente d'en insérer un second avec le même matricule :

```
INSERT INTO ELEVE (matricule, nom, sexe, date_naiss)
VALUES ('E001', 'MBALLA Jean', 'M', '15/03/2007');
-- ERREUR : violation de la contrainte de clé primaire.
-- La valeur 'E001' existe déjà : insertion rejetée.
```

Le SGBD refuse l'opération : sans cette contrainte, on aurait deux élèves différents portant le même matricule, et il deviendrait impossible de savoir à qui appartient chaque note du bulletin.

6.3 La contrainte de valeur autorisée (domaine)

Certains champs ne peuvent prendre leurs valeurs que dans un **ensemble fini et connu à l'avance**. La contrainte de **valeur autorisée** (aussi appelée contrainte de domaine) impose qu'une valeur appartienne à une liste ou à un intervalle précis.

EXEMPLE Exemples de valeurs autorisées

- le champ `sexe` d'un élève n'accepte que les valeurs `{M, F}`;
- le champ `niveau` n'accepte que `{2nde, 1ère, Tle}`;
- le champ `note` n'accepte que les réels de l'intervalle `[0 ; 20]`.

Ainsi, la saisie de `sexe = 'X'` ou de `note = 27` sera rejetée par le SGBD.

6.4 La contrainte de valeur nulle

Lorsqu'on crée un champ, on doit décider s'il est **obligatoire** ou **facultatif** :

DÉFINITION Valeur nulle et NOT NULL

Une **valeur nulle** (notée `NULL`) signifie absence de valeur : le champ n'a pas été renseigné. La contrainte `NOT NULL` imposée à un champ le rend obligatoire : tout enregistrement doit obligatoirement contenir une valeur pour ce champ. Sans cette contrainte, le champ est facultatif.

Par exemple, dans la table `ELEVE`, le nom doit être obligatoire (`NOT NULL`), tandis que le numéro de téléphone du parent peut rester facultatif.

ATTENTION NULL n'est ni zéro ni texte vide

Erreur classique de débutant : confondre les trois notions suivantes.

- NULL : absence de valeur (l'information est inconnue ou non saisie) ;
- 0 : la valeur numérique zéro (une information bien réelle : un élève peut avoir 0/20 à un devoir!) ;
- '' (texte vide) : une chaîne de caractères de longueur nulle, qui est quand même une valeur.

Un champ NOT NULL accepte 0 et '', mais refuse NULL. Retenir : NULL signifie « je ne sais pas », et non « rien » ou « zéro ».

6.5 La contrainte de valeur par défaut

DÉFINITION Valeur par défaut

La contrainte de **valeur par défaut** (DEFAULT) attribue automatiquement une valeur à un champ lorsque l'utilisateur ne le renseigne pas lors de l'insertion.

L'intérêt est double : on gagne du temps lors de la saisie et on évite les valeurs nulles indésirables. Par exemple, dans une base gérant les habitants d'une commune camerounaise, on peut déclarer que le champ pays vaut par défaut 'Cameroun' : seuls les rares cas étrangers devront être saisis explicitement.

EXEMPLE Insertion avec valeur par défaut

```
INSERT INTO HABITANT (nom, quartier)
VALUES ('NGONO Aline', 'Bastos');
-- Le champ pays n'est pas renseigné :
-- le SGBD lui affecte automatiquement la valeur 'Cameroun'.
```

6.6 La contrainte d'unicité

DÉFINITION Contrainte d'unicité

La contrainte d'**unicité** (UNIQUE) interdit que deux enregistrements d'une même table possèdent la même valeur pour un champ donné.

Toute clé primaire est unique, mais l'inverse est faux : un champ peut être unique sans être la clé primaire. Exemple typique : le **numéro de Carte Nationale d'Identité (CNI)** dans une table de citoyens. Ce numéro n'est pas forcément la clé primaire de la table, mais il doit être unique : deux camerounais ne peuvent jamais porter le même numéro de CNI.

ATTENTION Clé primaire et unicité : ne pas confondre

- Une table possède une seule clé primaire, mais peut avoir plusieurs champs déclarés **UNIQUE**;
- une clé primaire est toujours unique et non nulle ; un champ **UNIQUE** est unique mais peut, selon le SGBD, admettre une valeur nulle.

6.7 La contrainte de format de valeur

DÉFINITION Contrainte de format

La contrainte de **format de valeur** impose qu'une donnée respecte une forme précise : longueur, position des caractères, présence de symboles obligatoires, etc.

Exemples concrets dans le contexte camerounais :

- une date doit être saisie au format JJ/MM/AAAA (par exemple 15/03/2007) ;
- un numéro de téléphone camerounais doit comporter exactement **9 chiffres** (par exemple 690123456) ;
- un matricule d'élève peut être imposé sous la forme d'une lettre suivie de trois chiffres (E001, E002, ...).

Une saisie comme 2007-03-15 (format inversé) ou un téléphone à 8 chiffres sera rejetée.

6.8 La contrainte d'intégrité référentielle

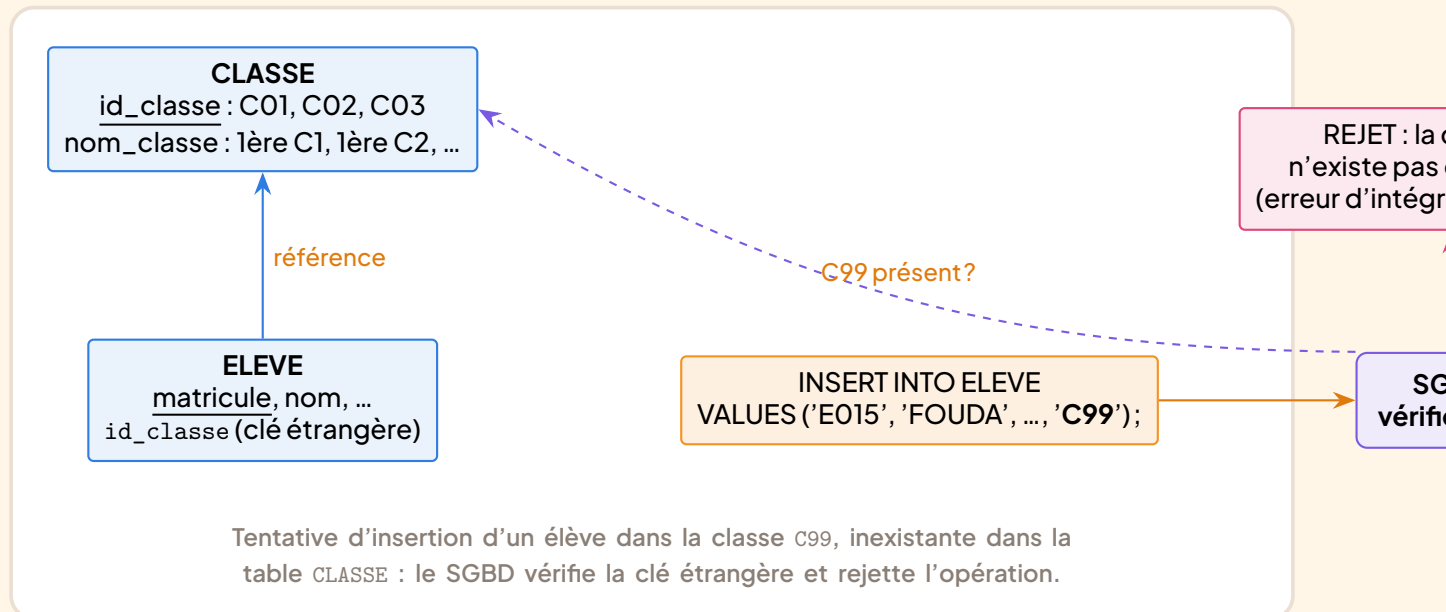
Cette contrainte concerne les **liens entre tables** et complète utilement le programme en vue du Baccalauréat.

DÉFINITION Intégrité référentielle

La contrainte d'**intégrité référentielle** impose que toute valeur d'une clé étrangère corresponde à une valeur existante de la clé primaire de la table référencée. Autrement dit : on ne peut pas faire référence à un enregistrement qui n'existe pas.

Concrètement : la table **ELEVE** contient une clé étrangère **id_classe** qui référence la clé primaire de la table **CLASSE**. Il est alors impossible d'inscrire un élève dans la classe C99 si cette classe

n'existe pas dans la table CLASSE.



6.9 Tableau récapitulatif des six contraintes

Contrainte	Exemple valide	Exemple rejeté
Clé primaire	matricule E002 (nouveau)	matricule E001 déjà pris
Valeur autorisée	sexe 'F'	sexe 'X'
Valeur nulle	nom 'ET0A' renseigné	nom NULL (champ obligatoire)
Valeur par défaut	pays omis → 'Cameroun'	— (pas de rejet : valeur automatique)
Unicité	CNI 102458796 nouvelle	CNI déjà attribuée à un autre
Format de valeur	tél. 690123456 (9 chiffres)	tél. 69012 (5 chiffres)

Les six contraintes d'intégrité au programme : pour chacune, un exemple accepté et un exemple rejeté par le SGBD.

6.10 Comment choisir les contraintes d'une table ?

MÉTHODE Les quatre questions à se poser pour chaque champ

Lors de la création d'une table, interroge-toi sur chaque champ en posant ces quatre questions, dans l'ordre :

1. Quel type? Nombre entier, nombre décimal, texte, date... (ex. : date_naiss est une date).

2. Obligatoire ou facultatif? Le champ peut-il rester vide? Si non, lui appliquer `NOT NULL` (ex. : le nom est obligatoire).
3. Unique? Deux enregistrements peuvent-ils partager la même valeur? Si non, appliquer `UNIQUE` (ex. : le numéro de CNI).
4. Valeur par défaut? Faut-il une valeur automatique en cas d'oubli de saisie? Si oui, appliquer `DEFAULT` (ex. : `pays = 'Cameroun'`).

On ajoute ensuite les contraintes de valeur autorisée et de format si le champ l'exige (sexe, note, téléphone...), sans oublier de désigner la clé primaire de la table.

EXERCICE RÉSOLU 6 Analyse d'insertions dans la table ELEVE

Énoncé. La table `ELEVE` est définie ainsi : `matricule` (clé primaire), `nom` (obligatoire), `sexe` (valeurs autorisées : M ou F), `telephone` (unique, 9 chiffres, facultatif), `pays` (par défaut 'Cameroun'). La table contient déjà l'enregistrement ('E001', 'ABENA Paul', 'M', '690111222', 'Cameroun'). Pour chacune des insertions suivantes, dire si elle est acceptée ou rejetée, et justifier :

1. ('E002', 'KAMGA Inès', 'F', '690111222', 'Cameroun')
2. ('E001', 'NDJOCK Marc', 'M', NULL, NULL)
3. ('E003', 'BELL Rita', 'X', '677888999', NULL)
4. ('E004', NULL, 'M', '655444333', NULL)
5. ('E005', 'TCHOUPPO Léa', 'F', NULL, NULL)

SOLUTION

Solution détaillée.

1. Rejetée : le téléphone 690111222 existe déjà (élève E001) : violation de la contrainte d'unicité.
2. Rejetée : le matricule E001 existe déjà : violation de l'unicité de la clé primaire.
3. Rejetée : le sexe 'X' n'appartient pas à l'ensemble {M, F} : violation de la contrainte de valeur autorisée.
4. Rejetée : le nom vaut NULL alors qu'il est obligatoire : violation de la contrainte de valeur nulle (`NOT NULL`).
5. Acceptée : le matricule est nouveau, le nom est renseigné, le sexe est valide, le téléphone est NULL (facultatif, donc autorisé) et le pays prend automatiquement la valeur par défaut 'Cameroun'.

EXPÉRIENCE TP : observer le rejet d'une insertion par le SGBD

Objectif : constater concrètement qu'un SGBD rejette les données qui violent une contrainte.

Logiciel : un SGBD simple (LibreOffice Base, MySQL/MariaDB ou SQLite).

Manipulation :

1. Créer la table `ELEVE` avec `matricule` comme clé primaire et `nom` en obligatoire ;
2. insérer l'élève ('E001', 'ABENA Paul') : l'insertion réussit ;
3. insérer à nouveau ('E001', 'Autre élève') ;
4. insérer un élève sans nom.

Observations : les étapes 3 et 4 échouent et le SGBD affiche un message d'erreur mentionnant la contrainte violée ; la table ne contient toujours qu'un seul enregistrement.

Interprétation : le SGBD a vérifié automatiquement les contraintes et protégé la cohérence de la base sans aucune intervention du programmeur.

LE SAVAIS-TU ? Un numéro de CNI unique pour chaque Camerounais

Grâce aux contraintes d'unicité appliquées dans les bases de données de l'administration, deux camerounais ne peuvent jamais recevoir le même numéro de Carte Nationale d'Identité. À chaque demande de carte, le système vérifie automatiquement que le nouveau numéro n'existe pas déjà. C'est le même principe qui garantit l'unicité des numéros matricules des candidats à l'examen du Probatoire ou des immatriculations de véhicules : la contrainte d'unicité, toute simple en apparence, est au cœur de la fiabilité des fichiers de l'État.

EXERCICE 6 Contraintes d'une table BIBLIOTHEQUE

La bibliothèque de ton lycée veut créer une table `LIVRE` avec les champs : `code_livre`, `titre`, `auteur`, `annee_parution`, `disponible` (oui/non). Pour chaque champ, réponds aux quatre questions de la méthode (type ? obligatoire ? unique ? valeur par défaut ?), choisis la clé primaire et justifie tes choix.

CORRIGÉ Exercice : table `LIVRE`

- `code_livre` : texte, obligatoire, unique ; c'est la clé primaire (elle identifie chaque livre de façon unique).
- `titre` : texte, obligatoire (NOT NULL), non unique (deux livres différents peuvent porter

le même titre).

- **auteur** : texte, obligatoire, non unique (un auteur peut avoir écrit plusieurs livres).
- **annee_parution** : nombre entier, facultatif (l'information peut manquer), avec une valeur autorisée raisonnable (par exemple entre 1900 et l'année en cours).
- **disponible** : valeurs autorisées {oui, non}, obligatoire, avec la valeur par défaut 'oui' (un livre nouvellement enregistré est a priori disponible).

À RETENIR L'essentiel sur les contraintes d'intégrité

- Une contrainte d'intégrité est une règle qui garantit l'exactitude et la cohérence des données ; le SGBD la vérifie automatiquement et rejette toute opération qui la viole.
- Les six contraintes au programme : clé primaire (unique et non nulle), valeur autorisée, valeur nulle (NOT NULL), valeur par défaut, unicité, format de valeur.
- L'intégrité référentielle impose qu'une clé étrangère corresponde à une clé primaire existante.
- NULL signifie « absence de valeur » : ce n'est ni zéro, ni un texte vide.

7 ACTIVITÉ D'INTÉGRATION

8 BASES DE DONNÉES RELATIONNELLES : INITIATION

8.1 Objectifs de l'activité

À l'issue de cette activité d'intégration, tu seras capable de :

- identifier les **tables** nécessaires à la gestion d'un lycée et décrire leurs éléments caractéristiques (**champs**, **enregistrements**, **clé primaire**, **clé étrangère**);
- définir pour chaque champ les **contraintes d'intégrité** adaptées (valeur nulle interdite, unicité, valeur par défaut, format, valeurs autorisées);
- décrire et simuler les **fonctions de définition** d'une base de données (création des tables, ajout d'une colonne, modification de type);
- décrire et simuler les quatre **fonctions de manipulation** : insertion, mise à jour, suppression, consultation;
- expliquer le rôle du **SGBD** qui rejette automatiquement les opérations violant une contrainte.

8.2 Situation

Le lycée de Nkongsamba compte 1250 élèves répartis dans 28 classes, de la 6^e à la Terminale. Jusqu'ici, la secrétaire, Mme Fotsing, gère les inscriptions et les paiements des frais scolaires dans de grands cahiers. Chaque rentrée, c'est le même cauchemar : des matricules attribués deux fois, des élèves inscrits dans des classes qui n'existent pas, des paiements introuvables et des heures perdues à établir la liste des élèves en règle.

Le proviseur décide de numériser cette gestion. Il te confie, à toi et à ton groupe, la conception de la base de données. Voici les informations recueillies auprès de Mme Fotsing :

- chaque **élève** possède un matricule unique de la forme NKB-2024-001, un nom, un prénom, une date de naissance, un sexe (M ou F) et appartient à une seule classe;
- chaque **classe** est identifiée par un code (6A, 1C, TD...), possède un libellé (« Première C »), une salle et un effectif maximal de 60 élèves;
- chaque **paiement** concerne un élève : le montant annuel des frais s'élève à 50000 FCFA, payable en plusieurs tranches; on enregistre la date et le montant de chaque versement;
- un élève nouvellement inscrit n'a encore rien payé : le total de ses versements vaut 0 FCFA par défaut.

Mme Fotsing signale aussi deux incidents récents à ne plus jamais revivre : l'élève de matricule NKB-2024-117 a été inscrit deux fois, et un paiement a été enregistré pour la classe 1Z, qui n'existe pas.

8.3 Consignes

Travail en groupes de 4 élèves. Durée : 2 heures. Matériel : papier, crayons, ou bien un SGBD (Access, MySQL, SQLite) si la salle informatique est disponible.

1. **Identifier les tables.** À partir de la situation, liste les tables nécessaires. Pour chacune, donne ses champs avec leur type (texte, nombre entier, date, montant) et précise la signification de chaque champ (dictionnaire des données).
2. **Choisir les clés.** Pour chaque table, choisis la clé primaire en justifiant ton choix. Place ensuite les clés étrangères qui relient les tables entre elles, et dessine le schéma relationnel (chaque table dans un cadre, les clés soulignées, les liens fléchés).
3. **Définir les contraintes.** Pour chaque champ, indique les contraintes d'intégrité : champ obligatoire ou valeur nulle autorisée, unicité, valeur par défaut, format de valeur (matricule), valeurs autorisées (sexe). Justifie chaque choix par une phrase issue de la situation.
4. **Peupler les tables.** Remplis chaque table avec 4 enregistrements fictifs mais réalistes et cohérents entre eux (les classes et élèves cités dans les paiements doivent exister).
5. **Simuler les manipulations.** Décris, en français puis avec la formulation SQL correspondante, les quatre opérations suivantes :
 - (a) inscrire un nouvel élève : ATEBA Larissa, née le 12/03/2008, en 1^{ère} C ;
 - (b) corriger un montant : le versement n° 3 était de 20000 FCFA et non 15000 FCFA ;
 - (c) supprimer l'élève NKB-2024-041, parti au lycée de Bafang ;
 - (d) consulter : lister les élèves de 1^{ère} C dont le total payé est inférieur à 50000 FCFA.
6. **Provoquer le rejet.** Tente volontairement deux insertions interdites : un élève avec le matricule NKB-2024-117 (déjà existant) et un paiement rattaché à la classe 1Z (inexistante). Explique pour chaque cas quelle contrainte est violée et comment le SGBD réagit.
7. **Faire évoluer la base.** Le proviseur veut désormais connaître le numéro de téléphone des parents. Quelle fonction de définition utilises-tu, et sur quelle table ?
8. **Présenter.** Chaque groupe expose son schéma relationnel au tableau ; la classe valide collectivement la cohérence des clés et des contraintes.

8.4 Ressources à mobiliser

DÉFINITION Base de données et SGBD

Une base de données (BD) est un ensemble structuré de données stockées de manière à être facilement accessibles, gérables et mises à jour. Un SGBD (Système de Gestion de Base de Données) est le logiciel qui permet de créer, manipuler, sécuriser et partager la base de données (ex. : MySQL, PostgreSQL, SQLite, Access, SQL Server, Oracle). Il garantit l'intégrité, la sécurité et la concurrence d'accès.

DÉFINITION Table, champ, enregistrement

Dans le modèle relationnel, une table (ou relation) est un tableau à deux dimensions. Les champs (colonnes, attributs) décrivent les propriétés des objets; chaque champ a un nom unique dans la table et un type de données. Les enregistrements (lignes, tuples) représentent une occurrence concrète de l'objet. L'ordre des lignes et des colonnes n'a pas d'importance théorique.

PROPRIÉTÉ Clés primaires et étrangères

- Clé primaire (Primary Key) : ensemble minimal de champs identifiant uniquement chaque enregistrement d'une table. Elle ne peut pas contenir de valeur nulle (NOT NULL) et ses valeurs sont uniques (UNIQUE). Notation : champ.
- Clé étrangère (Foreign Key) : champ (ou groupe de champs) d'une table qui référence la clé primaire d'une autre table (ou de la même table). Elle établit un lien (relation) entre les deux tables. La valeur d'une clé étrangère doit soit exister dans la table référencée, soit être nulle (si la relation est optionnelle).

PROPRIÉTÉ Fonctions de définition des données (LDD / DDL)

Ces commandes modifient la structure de la base :

- CREATE DATABASE / DROP DATABASE : créer / supprimer une base.
- CREATE TABLE / DROP TABLE : créer / supprimer une table.
- ALTER TABLE : ajouter (ADD), supprimer (DROP COLUMN), renommer (RENAME COLUMN) ou modifier le type (MODIFY / ALTER COLUMN) d'une colonne.

PROPRIÉTÉ Fonctions de manipulation des données (LMD / DML)

Ces commandes traitent le contenu des tables :

- INSERT INTO : ajouter un ou plusieurs enregistrements.
- UPDATE ... SET ... WHERE : modifier des valeurs existantes.
- DELETE FROM ... WHERE : supprimer des enregistrements.
- SELECT ... FROM ... WHERE : interroger (consulter) les données, avec jointures (JOIN), regroupements (GROUP BY), tris (ORDER BY).

PROPRIÉTÉ Contraintes d'intégrité principales

Contrainte	Rôle	Mot-clé SQL
Clé primaire	Identifiant unique, non nul	PRIMARY KEY
Clé étrangère	Intégrité référentielle	FOREIGN KEY ... REFERENCES
Non nullité	Valeur obligatoire	NOT NULL
Unicité	Pas de doublon (hors PK)	UNIQUE
Valeur par défaut	Valeur auto si omise	DEFAULT valeur
Valeurs autorisées	Domaine restreint (ex. M/F)	CHECK (sexe IN ('M','F'))
Format / Intervalle	Contrôle de forme ou bornes	CHECK (montant > 0)

À RETENIR Exemples de SGBD et domaines d'application

- SGBD relationnels : MySQL / MariaDB (Web, PME), PostgreSQL (robustesse, géo-spatial), SQLite (embarqué, mobile, local), Microsoft Access (bureautique, mono-utilisateur), SQL Server / Oracle (grandes entreprises, critiques).
- Domaines : gestion scolaire (bulletins, inscriptions), gestion commerciale (stocks, factures, clients), banque (comptes, transactions), santé (dossiers patients, rendez-vous), administration (état civil, impôts), télécoms (abonnés, facturation CAM-TEL/MTN/Orange).

8.5 Démarche et corrigé commenté**EXERCICE RÉSOLU 7** Corrigé commenté de l'activité

Concevoir la base de données du lycée de Nkongsamba en suivant les sept consignes.

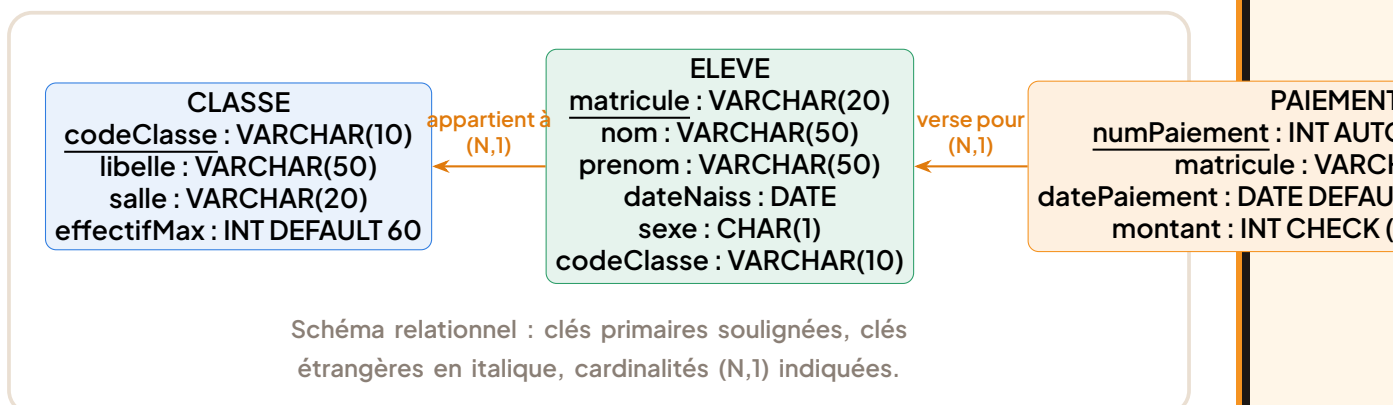
SOLUTION

Étape 1 : les tables et leurs champs (Dictionnaire des données). La situation mentionne trois entités fortes : les élèves, les classes, les paiements. On crée donc trois tables.

Table	Champ	Type et signification
CLASSE	codeClasse	Texte court (VARCHAR(10)), identifiant unique (1C, TD)
CLASSE	libelle	Texte (VARCHAR(50)), ex. « Première C »
CLASSE	salle	Texte (VARCHAR(20)), valeur nulle autorisée (salle non encore attribuée)
CLASSE	effectifMax	Entier (INT), valeur par défaut 60
ELEVE	matricule	Texte (VARCHAR(20)), format NKB-année-numéro
ELEVE	nom	Texte (VARCHAR(50)), obligatoire
ELEVE	prenom	Texte (VARCHAR(50)), obligatoire
ELEVE	dateNaiss	Date (DATE)
ELEVE	sexe	Caractère (CHAR(1)), M ou F
ELEVE	codeClasse	Texte (VARCHAR(10)), clé étrangère vers CLASSE
PAIEMENT	numPaielement	Entier (INT), auto-incrémenté (identifiant technique)
PAIEMENT	matricule	Texte (VARCHAR(20)), clé étrangère vers ELEVE
PAIEMENT	datePaielement	Date (DATE), défaut : date du jour (CURRENT_DATE)
PAIEMENT	montant	Entier (INT) ou Décimal, strictement positif (FCFA)

Étape 2 : les clés et schéma relationnel.

- **Clés primaires** : CLASSE.codeClasse (code métier stable), ELEVE.matricule (identifiant métier unique), PAIEMENT.numPaielement (identifiant technique auto-généré).
- **Clés étrangères** : ELEVE.codeClasse → CLASSE.codeClasse (un élève appartient à une classe); PAIEMENT.matricule → ELEVE.matricule (un paiement concerne un élève).



Étape 3 : les contraintes d'intégrité (justifiées par la situation).

Champ	Contraintes	Justification (Situation)
CLASSE.codeClasse	PK, NOT NULL, UNIQUE	Identifie chaque classe sans ambiguïté.
CLASSE.effectifMax	DEFAULT 60	« Effectif maximal de 60 élèves ».
CLASSE.salle	NULL autorisé	Une classe peut ne pas avoir de salle fixe au départ.
ELEVE.matricule	PK, NOT NULL, UNIQUE, CHECK (format)	« Matricule unique de la forme NKB-2024-001 »; évite le double emploi (NKB-2024-117).
ELEVE.nom, prenom	NOT NULL	Identité obligatoire.
ELEVE.sexe	CHECK (sexe IN ('M', 'F'))	« Sexe M ou F ».
ELEVE.codeClasse	FK, NOT NULL	« Appartient à une seule classe »; interdit la classe fantôme 1Z.
PAIEMENT.numPaielement	PK, AUTO_INCREMENT	Identifiant technique unique, géré par le SGBD.
PAIEMENT.matricule	FK, NOT NULL	Tout paiement concerne un élève existant.
PAIEMENT.montant	NOT NULL, CHECK (montant > 0)	Un versement a un montant positif.
PAIEMENT.datePaielement	DEFAULT CURRENT_DATE	Pratique pour la saisie rapide.

Étape 4 : peuplement cohérent (extraits).

Table CLASSE	Enregistrements
(codeClasse, libelle, salle, effectifMax)	('1C', 'Première C', 'S12', 60), ('1D', 'Première D', 'S14', 60), ('TC', 'Terminale C', 'S3', 55), ('2A', 'Seconde A', 'S7', 60)

Table ELEVE	Enregistrements
(matricule, nom, prenom, dateNaiss, sexe, codeClasse)	('NKB-2024-001', 'MBAPPE', 'André', '2008-01-15', 'M', '1C'), ('NKB-2024-041', 'NGO BELL', 'Sandra', '2008-06-02', 'F', '1C'), ('NKB-2024-117', 'TCHOUP0', 'Marc', '2007-09-22', 'M', '1D'), ('NKB-2024-205', 'FOTSO', 'Larissa', '2008-11-30', 'F', 'TC')

Table PAIEMENT	Enregistrements
(numPaiement, matricule, datePaiement, montant)	(1, 'NKB-2024-001', '2024-10-05', 25000), (2, 'NKB-2024-041', '2024-10-07', 50000), (3, 'NKB-2024-117', '2024-10-10', 15000), (4, 'NKB-2024-205', '2024-10-12', 20000)

Étape 5 : les quatre manipulations (SQL standard).

1. Insertion d'un nouvel élève (ATEBA Larissa, 12/03/2008, 1C). Le matricule suivant est NKB-2024-300.

```
INSERT INTO ELEVE (matricule, nom, prenom, dateNaiss, sexe, codeClasse)  
VALUES ('NKB-2024-300', 'ATEBA', 'Larissa', '2008-03-12', 'F', '1C');
```

2. Mise à jour du paiement n° 3 : montant passe de 15000 à 20000 FCFA.

```
UPDATE PAIEMENT  
SET montant = 20000  
WHERE numPaiement = 3;
```

3.

```
-- 1. Supprimer les paiements liés (si pas de cascade)
DELETE FROM PAIEMENT WHERE matricule = 'NKB-2024-041';
-- 2. Supprimer l'élève
DELETE FROM ELEVE WHERE matricule = 'NKB-2024-041';
```

4.

```
SELECT E.matricule, E.nom, E.prenom, COALESCE(SUM(P.montant), 0) AS total_paye
FROM ELEVE E
LEFT JOIN PAIEMENT P ON E.matricule = P.matricule
WHERE E.codeClasse = '1C'
GROUP BY E.matricule, E.nom, E.prenom
HAVING COALESCE(SUM(P.montant), 0) < 50000;
```

1.

2.

3.

4.

```
ALTER TABLE ELEVE
ADD COLUMN telParent VARCHAR(20); -- NULL par défaut (optionnel)
```

8.6 Bilan de l'activité

À RETENIR Synthèse : ce qu'il faut retenir

- Une base de données relationnelle se conçoit avant d'être saisie : identification des entités (tables), attributs (champs), identifiants (clés primaires) et liens (clés étrangères). C'est le passage du MCD (Modèle Conceptuel) au MLD (Modèle Logique).
- Les clés primaires garantissent l'unicité d'accès à chaque enregistrement (pas de doublon NKB-2024-117). Les clés étrangères garantissent l'intégrité référentielle (pas de classe 1Z fantôme).
- Les contraintes d'intégrité (PK, FK, NOT NULL, UNIQUE, CHECK, DEFAULT) sont déclarées une fois pour toutes dans la structure de la table. Le SGBD les vérifie automatiquement à chaque insertion/modification : c'est la garantie de la qualité des données.
- Le langage SQL sépare deux grands sous-ensembles :
 - LDD (DDL) : CREATE, ALTER, DROP → structure (définition).
 - LMD (DML) : INSERT, UPDATE, DELETE, SELECT → contenu (manipulation).
- Contrairement au cahier papier, la base relationnelle répond en millisecondes à des questions complexes : « Élèves de 1C non à jour de paiement », « Effectif réel par classe », « Total encaissé ce mois ».

MÉTHODE Concevoir une petite base relationnelle (approche MERISE simplifiée)

1. Analyser le énoncé : repérer les noms communs (entités) et les verbes (associations).
2. Lister les entités et leurs propriétés (attributs). Choisir un identifiant (clé primaire) par entité (naturel ou technique).
3. Repérer les associations (1 :N, N :M) et les transformer en clés étrangères du côté « N » (ou table de liaison si N :M).
4. Rédiger le dictionnaire des données (table, champ, type, contraintes).
5. Dessiner le schéma relationnel (cadre, soulignement PK, italique FK, flèches).
6. Écrire les scripts LDD (CREATE TABLE avec contraintes).
7. Tester avec des données (peuplement) et valider les manipulations (LMD).

ATTENTION Pièges fréquents au Probatoire

- Confondre PK et FK : La PK est dans la table « parente » (ex. CLASSE), la FK est dans la

table « fille »(ex. ELEVE).

- Oublier `NOT NULL` sur une FK obligatoire : Si un élève doit avoir une classe, `codeClasse` doit être `NOT NULL`. Par défaut, une FK autorise `NULL`.
- Type de données inadapté : Stocker une date en `VARCHAR` empêche les tris et calculs d'âge. Stocker un montant en `FLOAT` crée des erreurs d'arrondi (préférer `INT` en centimes/FCFA ou `DECIMAL(10,2)`).
- Ordre des suppressions : On ne peut pas supprimer un parent (CLASSE) si des enfants (ELEVE) existent, sauf `ON DELETE CASCADE` (dangereux) ou après avoir supprimé les enfants.
- Syntaxe `ALTER TABLE : ADD COLUMN` (PostgreSQL, SQLite, MySQL récent), `ADD` (MySQL ancien, Access), `MODIFY` vs `ALTER COLUMN` selon le SGBD.

LES AVAIS-TU ? Le Cameroun et les bases de données

Au Cameroun, les grands systèmes d'information reposent sur des SGBD robustes :

- MINEDUB / MINESEC : Gestion des effectifs, affectations, examens (Probatoire, Baccalauréat) via des applications centralisées (SIGE, plateforme d'inscription en ligne).
- CAMTEL / MTN / ORANGE : Bases d'abonnés (millions de lignes), facturation temps réel, gestion du réseau fibre (SIG).
- Banques (Afriland, BICEC, UBA, SCB) : Transactions ACID, cohérence stricte, haute disponibilité (Oracle, SQL Server).
- Élections (ELECAM) : Fichier électoral biométrique, dédoublement (contrainte d'unicité sur empreinte/IRIS).

La maîtrise du relationnel (SQL) est la compétence de base pour tout développeur, analyste ou administrateur système camerounais.

EXERCICE 7 Entraînement : La bibliothèque scolaire

Le CDI (Centre de Documentation et d'Information) de ton lycée veut informatiser la gestion des prêts.

- Un livre a un ISBN (clé), un titre, un auteur, une année, un nombre d'exemplaires total.
 - Un élève (utilise la table ELEVE existante) peut emprunter des livres.
 - Un emprunt lie un élève, un livre (un exemplaire), une date de début, une date de retour prévue, et une date de retour effective (null si pas encore rendu).
1. Propose le schéma relationnel (tables, PK, FK, cardinalités).
 2. Écris la requête `CREATE TABLE` pour la table EMPRUNT avec toutes les contraintes

utiles.

3. Écris la requête pour « Lister les livres actuellement non rendus (en retard) ».
4. Quelle contrainte empêche d'enregistrer un retour pour un emprunt qui n'existe pas ?

CORRIGÉ Exercice : La bibliothèque scolaire

1. Schéma :

- LIVRE (ISBN, titre, auteur, annee, nbExemplaires)
- ELEVE (matricule, nom, prenom, ...)
- EMPRUNT (numEmprunt, matricule → ELEVE, ISBN → LIVRE, dateDebut, datePrevue, dateRetourEffective)
- Cardinalités : ELEVE (1,N) – EMPRUNT ; LIVRE (1,N) – EMPRUNT.

2. LDD :

```
CREATE TABLE EMPRUNT (
    numEmprunt INT PRIMARY KEY AUTO_INCREMENT,
    matricule VARCHAR(20) NOT NULL,
    ISBN VARCHAR(13) NOT NULL,
    dateDebut DATE NOT NULL DEFAULT CURRENT_DATE,
    datePrevue DATE NOT NULL,
    dateRetourEffective DATE, -- NULL = pas encore rendu
    FOREIGN KEY (matricule) REFERENCES ELEVE(matricule),
    FOREIGN KEY (ISBN) REFERENCES LIVRE(ISBN),
    CHECK (datePrevue >= dateDebut),
    CHECK (dateRetourEffective IS NULL OR dateRetourEffective >= dateDebut)
);
```

3. Requête livres non rendus (en retard aujourd'hui) :

```
SELECT L.titre, L.ISBN, E.nom, E.prenom, EM.datePrevue
FROM EMPRUNT EM
JOIN ELEVE E ON EM.matricule = E.matricule
JOIN LIVRE L ON EM.ISBN = L.ISBN
WHERE EM.dateRetourEffective IS NULL
AND EM.datePrevue < CURRENT_DATE;
```

4. Contrainte : La clé étrangère FOREIGN KEY (numEmprunt) REFERENCES EMPRUNT(numEmprunt) (si on avait une table RETOUR) ou plus simplement ici, l'intégrité référentielle sur matricule et ISBN dans la table EMPRUNT empêche de créer un emprunt pour un élève ou un livre inexistant. Pour un retour, on fait un UPDATE sur la PK numEmprunt qui doit exister.

9 MÉTHODES ET EXERCICES RÉSOLUS

10 BASES DE DONNÉES RELATIONNELLES : INITIATION

10.1 Données, bases de données et SGBD

DÉFINITION Donnée et information

Une **donnée** est une représentation brute d'un fait, d'un objet ou d'un événement (texte, nombre, image, son) sans contexte particulier. Une **information** est une donnée traitée, structurée et contextualisée qui a une signification pour l'utilisateur et permet une prise de décision.

DÉFINITION Base de données

Une **base de données** (BD ou BDD) est un ensemble structuré, organisé et persistant de données relatives à un même domaine d'application, stockées sur un support informatique, accessibles à plusieurs utilisateurs et gérées par un logiciel dédié.

DÉFINITION SGBD

Un **SGBD** (Système de Gestion de Bases de Données) est un ensemble de logiciels qui permet de créer, décrire (définir), manipuler (interroger, modifier), administrer et protéger une base de données. Il assure l'indépendance entre les programmes et les données, la cohérence (intégrité), la sécurité (droits d'accès), la concurrence (accès simultané) et la sauvegarde/restauration.

PROPRIÉTÉ Fonctions principales d'un SGBD

Un SGBD offre quatre grandes familles de fonctions :

1. Fonctions de définition (LDD – Langage de Définition de Données) : créer/supprimer la base, créer/supprimer/modifier la structure des tables (colonnes, types, clés, index).
2. Fonctions de manipulation (LMD – Langage de Manipulation de Données) : insérer,

modifier, supprimer, consulter (interroger) les enregistrements.

3. Fonctions de contrôle (LCD – Langage de Contrôle de Données) : gérer les droits d'accès (GRANT, REVOKE), l'intégrité transactionnelle (COMMIT, ROLLBACK).
4. Fonctions d'administration : sauvegarde, restauration, optimisation (index, statistiques), surveillance de l'espace disque, réplication.

EXEMPLE Du cahier de notes au SGBD

Un professeur note les devoirs sur un cahier (support papier, accès séquentiel, pas de recherche rapide, risque de perte). Il transpose ces données dans un tableur (fichier plat, redondance si plusieurs classes, pas de contrôle d'accès). Enfin, il utilise un SGBD (ex. MySQL) : les notes sont stockées une seule fois, accessibles par l'administration, les professeurs et les élèves (selon droits), avec contrôle de cohérence (note entre 0 et 20) et sauvegarde automatique.

10.2 Composants d'un SGBD

DÉFINITION Dictionnaire de données (Catalogue système)

Le **dictionnaire de données** (ou catalogue système) est la partie de la base qui stocke les métadonnées : description de la structure des tables (noms, types, tailles, clés, contraintes), des vues, des index, des utilisateurs et de leurs privilèges. Le SGBD le consulte pour valider chaque requête.

DÉFINITION Moteur de stockage

Le **moteur de stockage** gère l'organisation physique des données sur le disque (fichiers, pages, blocs), les buffers mémoire (cache), les index (B-arbres, tables de hachage) et l'exécution des algorithmes d'accès (parcours séquentiel, parcours d'index).

DÉFINITION Processeur de requêtes

Le **processeur de requêtes** analyse une requête SQL (analyse syntaxique, sémantique), l'optimise (choix du plan d'exécution le moins coûteux en I/O et CPU) et l'exécute en pilotant le moteur de stockage.

DÉFINITION Gestionnaire de transactions

Le **gestionnaire de transactions** garantit les propriétés **ACID** : Atomicité (tout ou rien), Cohérence (respect des contraintes), Isolation (exécution concurrente sans interférence), Durabilité (persistance après validation).

À RETENIR Architecture globale d'un SGBD

Composant	Rôle
Dictionnaire de données	Mémoire la structure (métadonnées)
Moteur de stockage	Accès physique aux données (disque/RAM)
Processeur de requêtes	Analyse, optimise, exécute le SQL
Gestionnaire de transactions	Garantit ACID, verrous, journal (log)
Gestionnaire de sécurité	Authentification, autorisations (GRANT/REVOKE)
Outils d'administration	Backup, restore, import/export, monitoring

10.3 Tables, champs, enregistrements et clés

DÉFINITION Table (Relation)

Une **table** (ou **relation** en terminologie formelle) est une structure de données bidimensionnelle qui représente une entité du monde réel (élève, livre, produit). Elle porte un nom unique dans la base et est composée de **champs** (colonnes) et d'**enregistrements** (lignes ou tuples).

DÉFINITION Champ (Attribut / Colonne)

Un **champ** correspond à une propriété de l'entité (ex. nom, dateNaissance). Il est défini par un nom unique dans la table, un type de données (entier, réel, texte, date, booléen, binaire) et éventuellement des contraintes (clé primaire, non nul, unique, défaut, check).

DÉFINITION Enregistrement (Tuple / Ligne)

Un **enregistrement** est une occurrence concrète de l'entité : un ensemble ordonné de va-

leurs, une pour chaque champ de la table. L'ordre des lignes n'a pas de signification théorique (ensemble non ordonné).

DÉFINITION Clé primaire

La **clé primaire** (Primary Key – PK) est un champ (ou une combinaison de champs) qui identifie de manière unique chaque enregistrement de la table. Elle respecte deux règles strictes : unicité (pas de doublons) et non-nullité (valeur obligatoire). On la note soulignée dans le schéma relationnel : $\text{TABLE}(\underline{\text{champClé}}, \dots)$.

DÉFINITION Clé étrangère

Une **clé étrangère** (Foreign Key – FK) est un champ (ou groupe de champs) d'une table T_2 qui référence la clé primaire d'une autre table T_1 . Elle crée un lien (relation) entre les deux tables. Dans T_2 , elle autorise les doublons (plusieurs lignes de T_2 peuvent pointer vers la même ligne de T_1) et peut être nulle si le lien est optionnel. On la note avec un dièse # ou une étoile : $T_2(\underline{\text{pk}}, \text{fk}\#, \dots)$.

MÉTHODE Décrire les éléments d'une table

Pour décrire rigoureusement une table dans une copie d'examen :

1. Nommer la table et préciser l'entité modélisée (élève, produit, emprunt...).
2. Lister les champs (colonnes) : pour chacun, donner le nom, le type de données (entier, texte, date, réel...) et son rôle.
3. Compter les enregistrements (lignes) : chaque ligne = une occurrence de l'entité.
4. Identifier la clé primaire : champ (ou ensemble) **unique** et **non nul**. On la souligne.
5. Repérer les clés étrangères : champ référençant la clé primaire d'une autre table ; il relie les tables.

Réflexe : une clé primaire se choisit parmi les champs dont les valeurs ne se répètent jamais (matricule, numéro, code). Un nom ou une date de naissance ne convient pas (doublons possibles).

EXERCICE RÉSOLU 8 Analyse de la table d'une bibliothèque scolaire

La bibliothèque du lycée de Bafoussam gère ses livres dans la table `LIVRE` suivante :

codeLivre	titre	auteur	anneeEdition
L001	Les Misérables	Victor Hugo	2015
L002	Une si longue lettre	Mariama Bâ	2018
L003	Le Petit Prince	Antoine de Saint-Exupéry	2015

1. Donner les champs de cette table.
2. Combien d'enregistrements contient-elle ?
3. Quel champ peut servir de clé primaire ? Justifier.
4. Pourquoi le champ `anneeEdition` ne peut-il pas être clé primaire ?

SOLUTION

Solution détaillée.

1. La table `LIVRE` possède quatre champs : `codeLivre` (identifiant, texte), `titre` (texte), `auteur` (texte) et `anneeEdition` (entier).
2. La table contient trois enregistrements (trois lignes), correspondant aux trois livres L001, L002 et L003.
3. Le champ `codeLivre` peut servir de clé primaire, car il vérifie les deux conditions : valeurs uniques (L001, L002, L003 distincts) et non nulles (chaque livre a un code). Schéma : `LIVRE(codeLivre, titre, auteur, anneeEdition)`.
4. Le champ `anneeEdition` contient des doublons : L001 et L003 ont tous deux 2015. Une clé primaire doit identifier chaque enregistrement de manière unique.

Conclusion : la table `LIVRE` a 4 champs, 3 enregistrements, clé primaire `codeLivre`.

MÉTHODE Repérer et justifier une clé étrangère

1. Repérer dans une table T_2 un champ qui a le même nom (ou même signification) que la clé primaire d'une table T_1 .
2. Vérifier qu'il est clé primaire dans T_1 (valeurs uniques) mais pas dans T_2 (doublons autorisés).
3. Conclure : ce champ est une **clé étrangère** dans T_2 , référençant la PK de T_1 .
4. Préciser le lien : chaque enregistrement de T_2 est associé à un enregistrement de T_1 .

Réflexe : notation schématique : `EMPRUNT(numEmprunt, dateEmprunt, codeLivre#, matriculeEleve#)`.

EXERCICE RÉSOLU 9 Clés dans la base d'une bibliothèque

La bibliothèque utilise deux tables : ELEVE(matricule, nom, classe) EMPRUNT(numEmprunt, dateEmprunt, matricule#)

1. Quelle est la clé primaire de EMPRUNT ?
2. Identifier la clé étrangère de EMPRUNT et son rôle.
3. Un même élève peut-il apparaître plusieurs fois dans EMPRUNT ? Justifier.

SOLUTION

Solution détaillée.

1. La clé primaire de EMPRUNT est numEmprunt : chaque emprunt a un numéro unique et non nul.
2. La clé étrangère est matricule : ce n'est pas une PK dans EMPRUNT, mais il référence la PK matricule de ELEVE. Son rôle : relier les deux tables pour savoir quel élève a emprunté.
3. Oui, matricule n'étant pas PK dans EMPRUNT, ses valeurs peuvent se répéter : un élève qui emprunte trois livres apparaît dans trois lignes distinctes (trois numEmprunt différents).

Conclusion : la clé étrangère matricule matérialise le lien « un élève peut effectuer plusieurs emprunts ».

10.4 Fonctions de définition d'une base de données (LDD)**PROPRIÉTÉ** Principales instructions LDD (SQL standard)

- CREATE DATABASE nom_base ; - crée la base.
- DROP DATABASE nom_base ; - supprime la base (irréversible).
- CREATE TABLE nom_table (col1 type1 [contraintes], col2 type2, ..., PRIMARY KEY (col)) ; - crée une table.
- DROP TABLE nom_table ; - supprime la table et ses données.
- ALTER TABLE nom_table ADD nom_colonne type [contraintes] ; - ajoute une colonne.
- ALTER TABLE nom_table DROP COLUMN nom_colonne ; - supprime une colonne.
- ALTER TABLE nom_table RENAME COLUMN ancien_nom TO nouveau_nom ; - renomme (standard SQL, syntaxe MySQL 8.0+ / PostgreSQL).
- ALTER TABLE nom_table MODIFY COLUMN nom_colonne nouveau_type ; - change le

type (syntaxe MySQL/Oracle ; PostgreSQL : ALTER COLUMN ... TYPE).

Note : les noms (base, table, colonnes) s'écrivent sans espaces ni accents ; le point-virgule termine l'instruction.

MÉTHODE Reconnaître et rédiger les fonctions de définition (LDD)

1. Identifier l'opération : création/suppression base, création/suppression table, ajout/renomme/suppression colonne, modification type.
2. Associer l'ordre SQL correspondant (voir propriété ci-dessus).
3. Rédiger la requête : syntaxe exacte, noms valides, point-virgule final.

EXERCICE RÉSOLU 10 Création de la base d'un cybercafé

Le gérant d'un cybercafé à Douala veut créer la base CYBERCAFE avec une table CLIENT : idClient (entier, PK), nom (texte 50), telephone (texte 15). Puis ajouter solde (montant décimal).

SOLUTION

Solution détaillée.

Étape 1 : création de la base.

```
CREATE DATABASE CYBERCAFE;
```

Étape 2 : création de la table.

```
CREATE TABLE CLIENT (  
  idClient INT,  
  nom VARCHAR(50),  
  telephone VARCHAR(15),  
  PRIMARY KEY (idClient)  
);
```

Étape 3 : ajout de la colonne solde. Pour un montant d'argent, on préfère DECIMAL(10,2) (exact) à REAL (approché).

```
ALTER TABLE CLIENT ADD solde DECIMAL(10,2);
```

Conclusion : ces requêtes relèvent de la définition (LDD) : elles modifient la structure, pas le contenu.

10.5 Fonctions de manipulation d'une base de données (LMD)

PROPRIÉTÉ Quatre instructions LMD fondamentales

1. **Insertion**: `INSERT INTO table (col1, col2, ...) VALUES (val1, val2, ...) ;`
2. **Mise à jour**: `UPDATE table SET col = nouvelle_valeur WHERE condition ;`
3. **Suppression**: `DELETE FROM table WHERE condition ;`
4. **Consultation**: `SELECT col1, col2 FROM table WHERE condition ;`

Règles de syntaxe : textes entre apostrophes simples '...'; nombres sans apostrophes; dates 'AAAA-MM-JJ'; WHERE obligatoire dans UPDATE et DELETE (sinon toute la table); SELECT ne modifie jamais les données.

MÉTHODE Reconnaître et rédiger les fonctions de manipulation (LMD)

1. Identifier l'action : ajouter (INSERT), modifier (UPDATE), supprimer (DELETE), lire (SELECT).
2. Écrire la requête en respectant l'ordre des clauses et le typage des valeurs.
3. Vérifier la présence de WHERE pour UPDATE/DELETE ciblés.

EXERCICE RÉSOLU 11 Manipulation de la table CLIENT

Table `CLIENT(idClient, nom, telephone, solde)`.

1. Insérer le client (1, 'Ngo Bell', '699112233', 2500.00).
2. Augmenter de 500 FCFA le solde du client n°1.
3. Afficher nom et solde des clients avec solde < 1000 FCFA.
4. Supprimer le client n°7.

SOLUTION

Solution détaillée.

1. Insertion :

```
INSERT INTO CLIENT (idClient, nom, telephone, solde)
VALUES (1, 'Ngo Bell', '699112233', 2500.00);
```

2. Mise à jour :

```
UPDATE CLIENT SET solde = solde + 500 WHERE idClient = 1;
```

Nouveau solde = 3000.00 FCFA.

3. Consultation :

```
SELECT nom, solde FROM CLIENT WHERE solde < 1000;
```

4. Suppression :

```
DELETE FROM CLIENT WHERE idClient = 7;
```

Sans WHERE, la table serait vidée.

10.6 Contraintes d'intégrité

DÉFINITION Contrainte d'intégrité

Une **contrainte d'intégrité** est une règle déclarée dans la structure de la table que le SGBD vérifie automatiquement à chaque insertion ou modification. Elle garantit la **cohérence** (respect des règles de gestion) et la **validité** des données.

PROPRIÉTÉ Contraintes d'intégrité courantes (SQL)

Contrainte	Mot-clé SQL	Règle / Exemple
Clé primaire	PRIMARY KEY	Unique + Non nul (identifiant)
Clé étrangère	FOREIGN KEY ... REFERENCES	Valeur existant dans la table parente (ou NULL)
Unicité	UNIQUE	Pas de doublons (ex. email, téléphone)
Non nul	NOT NULL	Champ obligatoire
Valeur par défaut	DEFAULT valeur	Valeur auto si omise (ex. DEFAULT 1)
Valeur autorisée (Domaine)	CHECK (condition)	Intervalle ou ensemble (ex. note BETWEEN 0 AND 20)
Format / Masque	CHECK (col LIKE motif)	Forme imposée (ex. matricule '___-___')

MÉTHODE Identifier une contrainte d'intégrité

Pour chaque règle de gestion énoncée :

1. Clé primaire : « identifiant unique » → PRIMARY KEY.
2. Unicité : « pas deux fois la même valeur » (hors PK) → UNIQUE.
3. Non nul : « champ obligatoire / doit être renseigné » → NOT NULL.
4. Défaut : « si non précisé, vaut X » → DEFAULT X.
5. Domaine (Check) : « valeur comprise entre / appartenant à » → CHECK.
6. Format : « forme précise (masque) » → CHECK ... LIKE.
7. Clé étrangère : « référence à une autre table » → FOREIGN KEY ... REFERENCES.

Réflexe : à l'examen, citer la contrainte et la justifier par la règle de gestion.

EXERCICE RÉSOLU 12 Contraintes sur la table des notes d'un lycée

Table NOTE(idNote, matriculeEleve, matiere, valeur, trimestre). Règles :

- Chaque note a un identifiant unique.
- Le matricule de l'élève est obligatoire.
- La note est entre 0 et 20.
- Trimestre vaut 1 par défaut.
- Deux élèves n'ont pas le même matricule (règle sur la table ELEVE, pas NOTE).

SOLUTION

Solution détaillée.

Association règle → contrainte (sur la table NOTE) :

- « identifiant unique » ⇒ Clé primaire sur idNote (PRIMARY KEY).
- « matricule obligatoire » ⇒ Non nul (NOT NULL).
- « note entre 0 et 20 » ⇒ Valeur autorisée (CHECK (valeur >= 0 AND valeur <= 20)).
- « trimestre = 1 par défaut » ⇒ Valeur par défaut (DEFAULT 1).
- « matricule unique par élève » ⇒ Unicité (UNIQUE) sur la table ELEVE, pas sur NOTE (un élève a plusieurs notes).

Requête de création de la table NOTE (avec FK vers ELEVE) :

```
CREATE TABLE NOTE (  
  idNote INT PRIMARY KEY,  
  matriculeEleve VARCHAR(10) NOT NULL, -- obligatoire
```



```

matiere VARCHAR(30),
valeur DECIMAL(4,2) CHECK (valeur >= 0 AND valeur <= 20), -- intervalle
trimestre INT DEFAULT 1, -- défaut
FOREIGN KEY (matriculeEleve) REFERENCES ELEVE(matricule) -- lien
);

```

Vérification : insertion d'une note 25 → rejet (CHECK) ; omission trimestre → 1 enregistré (DEFAULT) ; matricule inexistant dans ELEVE → rejet (FOREIGN KEY).

Conclusion : les contraintes sont déclarées à la création ; le SGBD les applique automatiquement.

10.7 SGBD du marché et domaines d'application

MÉTHODE Citer des SGBD et des domaines d'application

1. SGBD à connaître : MySQL / MariaDB (libre, web), PostgreSQL (libre, avancé), Oracle Database (entreprise, critique), Microsoft SQL Server (environnement Microsoft), SQLite (embarqué, mobile, local), Microsoft Access (bureautique, mono-utilisateur).
2. Domaines d'application (exemples Cameroun) :
 - Éducation : gestion des notes, bulletins, inscriptions (ex. plateforme OBC, résultats Probatoire).
 - Santé : dossiers patients, rendez-vous, stock pharmacie (hôpitaux, cliniques).
 - Banque / Finance : comptes clients, transactions, crédits (Afriland First Bank, BI-CEC, UBA).
 - Commerce / Stock : articles, ventes, fournisseurs (boutiques, supermarchés, jumia.cm).
 - Administration : état civil, impôts, cartes d'identité, fonction publique.
 - Télécoms : abonnés, numéros, consommation, recharge (MTN, Orange, CAMTEL, Nexttel).
3. Distinction : le SGBD est le logiciel (moteur) ; la base de données est le contenu structuré (données + schéma).

EXERCICE RÉSOLU 13 SGBD et applications au Cameroun

1. Définir base de données et SGBD.
2. Citer quatre SGBD.

3. Trois domaines d'application au Cameroun avec exemple de données.

SOLUTION

Solution détaillée.

1. Base de données : ensemble structuré, organisé, persistant de données d'un même domaine, accessible en partage. SGBD : logiciel permettant de créer, décrire, manipuler, administrer et sécuriser une base de données.
2. Quatre SGBD : MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server (SQLite, Access acceptés).
3. Trois domaines :
 - Éducation : MINSEC / OBC gère candidats, centres, notes, délibérations.
 - Banque : Afriland / BICEC gère comptes, virements, cartes, historiques.
 - Télécoms : MTN / Orange / CAMTEL gère abonnés (MSISDN, IMSI), crédit, data, forfaits.

LESAVAIS-TU ? Histoire : Edgar Codd et le modèle relationnel

En 1970, le mathématicien britannique Edgar Frank Codd (IBM San Jose) publie « A Relational Model of Data for Large Shared Data Banks ». Il propose de remplacer les modèles hiérarchiques et en réseau (complexes, navigation par pointeurs) par des relations (tables) manipulées par un langage ensembliste de haut niveau. C'est la naissance du modèle relationnel. Le langage SQL (Structured Query Language), initialement SEQUEL, est développé par IBM (System R) à la fin des années 70, standardisé ANSI/ISO en 1986. Codd reçoit le prix Turing en 1981. Au Cameroun, l'enseignement du relationnel et du SQL s'est généralisé dans les lycées (séries C, D, E, F, G) depuis les programmes 2010, remplaçant l'approche fichiers plats.

EXERCICE 8 Entraînement – Probatoire Blanc

Sujet : Gestion d'un stock de fournitures scolaires dans une librairie de Yaoundé. On souhaite créer une base LIBRAIRIE avec deux tables :

- FOURNISSEUR(idFourn, nomFourn, ville, telephone)
- PRODUIT(codeProd, designation, prixUnitaire, quantiteStock, idFourn#)

Règles : idFourn et codeProd sont des identifiants uniques ; le téléphone du fournisseur est unique ; le prix est positif ; la quantité en stock ne peut pas être négative ; si la quantité n'est pas saisie, elle vaut 0 ; tout produit provient d'un fournisseur existant.

1. Écrire la requête CREATE DATABASE.

2. Écrire les deux requêtes `CREATE TABLE` avec toutes les contraintes (PK, FK, NOT NULL, UNIQUE, CHECK, DEFAULT).
3. Écrire l'`INSERT` pour le produit : ('STYLO-BIC', 'Stylo bille bleu', 150, 500, 1).
4. Écrire l'`UPDATE` pour augmenter de 10% le prix des produits du fournisseur n°1.
5. Écrire le `SELECT` pour afficher la désignation et le stock des produits en rupture (quantité = 0).
6. Écrire le `DELETE` pour supprimer le produit de code 'VIEUX-STOCK'.

CORRIGÉ Exercice – Gestion stock librairie

```
CREATE DATABASE LIBRAIRIE;
```

```
CREATE TABLE FOURNISSEUR (  
    idFourn INT PRIMARY KEY,  
    nomFourn VARCHAR(50) NOT NULL,  
    ville VARCHAR(30),  
    telephone VARCHAR(15) UNIQUE  
);  
  
CREATE TABLE PRODUIT (  
    codeProd VARCHAR(20) PRIMARY KEY,  
    designation VARCHAR(50) NOT NULL,  
    prixUnitaire DECIMAL(10,2) CHECK (prixUnitaire > 0),  
    quantiteStock INT DEFAULT 0 CHECK (quantiteStock >= 0),  
    idFourn INT NOT NULL,  
    FOREIGN KEY (idFourn) REFERENCES FOURNISSEUR(idFourn)  
);
```

```
INSERT INTO PRODUIT (codeProd, designation, prixUnitaire, quantiteStock, idFourn)  
VALUES ('STYLO-BIC', 'Stylo bille bleu', 150, 500, 1);
```

```
UPDATE PRODUIT SET prixUnitaire = prixUnitaire * 1.10 WHERE idFourn = 1;
```

```
SELECT designation, quantiteStock FROM PRODUIT WHERE quantiteStock = 0;
```

```
DELETE FROM PRODUIT WHERE codeProd = 'VIEUX-STOCK';
```

ATTENTION Les 5 erreurs qui coûtent des points au Probatoire

1. Confondre champ et enregistrement. Champ = colonne (caractéristique); Enregistrement = ligne (occurrence). Compter les colonnes pour donner le nombre d'enregistrements = erreur éliminatoire.
2. Choisir comme clé primaire un champ à doublons. Nom, prénom, date de naissance, ville : doublons fréquents. Une PK doit être unique et non nulle → toujours justifier ces deux propriétés.
3. Oublier WHERE dans UPDATE ou DELETE. Sans condition, tous les enregistrements sont modifiés/supprimés. Écrire explicitement : « La clause WHERE limite l'action à l'enregistrement ciblé. »
4. Confondre LDD et LMD. CREATE/ALTER/DROP = structure (LDD); INSERT/UPDATE/DELETE/SELECT = contenu (LMD). Classer la requête avant de répondre.
5. Mauvaise gestion des apostrophes et types. Texte → '...'; Nombre → sans apostrophes; Date → 'AAAA-MM-JJ'. VALUES (1, Ngo Bell, ...) sans apostrophes sur le nom = erreur syntaxe. Vérifier le point-virgule final et l'absence d'accents dans les identificateurs.

11 EXERCICES

11.1 Je vérifie mes connaissances

EXERCICE 9 QCM de synthèse

Pour chaque question, une seule réponse est correcte. Recopie la lettre correspondante sur ta feuille.

Question 1. Un SGBD est :

- a) un ensemble de données organisées sur un support informatique ;
- b) un logiciel qui permet de créer, de décrire, de manipuler et d'administrer des bases de données ;
- c) une ligne d'une table ;
- d) un langage de programmation.

Question 2. Dans une table, un enregistrement correspond à :

- a) une colonne ;
- b) une ligne ;
- c) la clé primaire ;
- d) le nom de la table.

Question 3. La clé étrangère d'une table sert à :

- a) identifier de manière unique chaque enregistrement de cette table ;
- b) interdire les valeurs nulles ;
- c) établir un lien avec la clé primaire d'une autre table ;
- d) crypter les données.

Question 4. Parmi les logiciels suivants, lequel est un SGBD ?

- a) Microsoft Word ;
- b) MySQL ;
- c) Mozilla Firefox ;
- d) Windows 11.

Question 5. La différence entre une valeur nulle et une valeur par défaut est :

- a) il n'y a aucune différence ;
- b) la valeur nulle signifie « absence de valeur », tandis que la valeur par défaut est une valeur attribuée automatiquement si l'utilisateur n'en saisit pas ;
- c) la valeur nulle est toujours égale à zéro ;

d) la valeur par défaut est toujours vide.

EXERCICE 10 Vrai ou faux, avec justification

Pour chacune des affirmations suivantes, indique si elle est vraie ou fausse, puis justifie ta réponse en une ou deux phrases.

1. Deux enregistrements d'une même table peuvent avoir la même valeur de clé primaire.
2. Une table peut contenir plusieurs clés étrangères.
3. Supprimer une colonne d'une table est une fonction de manipulation d'une base de données.
4. Un champ déclaré avec la contrainte « valeur nulle interdite » peut rester vide lors de la saisie.
5. Microsoft Access, PostgreSQL et Oracle Database sont des exemples de SGBD.

EXERCICE 11 Définitions

Définis chacun des termes suivants en une phrase claire et précise, puis donne un exemple tiré de la gestion des élèves de ton lycée :

1. Donnée ;
2. Base de données ;
3. Table ;
4. Champ ;
5. Enregistrement ;
6. Clé primaire.

EXERCICE 12 Lecture directe d'une table

On considère la table `ELEVE` ci-dessous, utilisée par le secrétariat d'un lycée de Yaoundé :

matricule	nom	sexe	date_naiss	#id_classe
E001	Ngo Bell Amina	F	12/03/2007	C01
E002	Essomba Jean	M	25/07/2006	C01
E003	Fotso Larissa	F	03/11/2007	C02
E004	Mbarga Paul	M	18/01/2007	C02
E005	Ngo Bell Amina	F	30/05/2008	C03
E006	Kamdem Steve	M	09/09/2006	C03

Réponds aux questions suivantes :

1. Combien cette table contient-elle de champs ? De combien d'enregistrements ?
2. Quel champ joue le rôle de clé primaire ? Justifie ta réponse.
3. Le champ `#id_classe` est une clé étrangère : que représente-t-elle et à quelle autre table renvoie-t-elle probablement ?
4. Justifie pourquoi le champ `nom` ne peut pas être choisi comme clé primaire de cette table.

11.2 Je m'entraîne

EXERCICE 13 Définition ou manipulation ?

Le proviseur d'un collège de Bafoussam fait appel à un informaticien pour gérer la base de données de l'établissement. Classe chacune des opérations suivantes dans la catégorie qui convient : fonction de définition ou fonction de manipulation de la base de données. Présente ta réponse sous forme d'un tableau à deux colonnes.

1. Créer la table `ELEVE` ;
2. Insérer un nouvel élève dans la table `ELEVE` ;
3. Ajouter une colonne `telephone_parent` à la table `ELEVE` ;
4. Modifier la note d'informatique d'un élève ;
5. Supprimer la base de données de l'année précédente ;
6. Renommer la colonne `date_naiss` en `naissance` ;
7. Consulter la liste des élèves absents au cours de la semaine ;
8. Changer le type du champ `moyenne` (passer d'un entier à un nombre décimal).

EXERCICE 14 Quelle contrainte est violée ?

La base de données d'un lycée de Douala contient la table `ELEVE(matricule, nom, sexe, date_naiss, #id_classe)`. Pour chacune des situations suivantes, cite la contrainte d'intégrité violée (clé primaire, valeur nulle, valeur autorisée, format de valeur, unicité de valeur, valeur par défaut, intégrité référentielle (clé étrangère)) et explique en une phrase pourquoi la saisie est refusée.

1. On tente d'enregistrer un élève avec le matricule `E001` alors qu'un autre élève possède déjà ce matricule.
2. On saisit un élève en laissant le champ `nom` vide, alors que ce champ est obligatoire.
3. On saisit la date de naissance « 32/15/2020 ».
4. On saisit la valeur « X » dans le champ `sexe`, qui n'accepte que « M » ou « F ».
5. On rattache un élève à la classe `C09` qui n'existe pas dans la table `CLASSE`.
6. On saisit deux fois le numéro de téléphone « 699 00 00 00 » dans un champ déclaré unique.

EXERCICE 15 Schéma relationnel d'une bibliothèque scolaire

La bibliothèque d'un lycée de Garoua souhaite informatiser la gestion de ses prêts. On retient les informations suivantes : chaque livre possède un code, un titre, un auteur et une année d'édition ; chaque emprunteur possède un numéro de carte, un nom et une classe ; chaque prêt concerne un emprunteur et un livre, et comporte une date de prêt et une date de retour prévue.

1. Propose trois tables : `LIVRE`, `EMPRUNTEUR` et `PRET`, en précisant pour chacune la liste de ses champs.
2. Souligne la clé primaire de chaque table et fais précéder chaque clé étrangère du symbole #.
3. Écris le schéma relationnel complet de cette base de données.
4. Explique pourquoi la table `PRET` contient deux clés étrangères.

EXERCICE 16 Composants et exemples de SGBD

1. Cite quatre composants d'un SGBD et décris en une phrase le rôle de chacun.
2. Cite quatre exemples de SGBD, en précisant pour chacun s'il est libre ou propriétaire.
3. Cite quatre domaines d'application des bases de données dans la vie quotidienne au

Cameroun (par exemple : banques, hôpitaux, écoles, opérateurs de téléphonie mobile) et précise pour chacun un exemple de données stockées.

11.3 Je me prépare au Probatoire

EXERCICE 17 Facturation d'une boutique de Douala

Madame Etoundi tient une boutique d'articles scolaires à Douala. Pour suivre ses ventes, elle souhaite créer une base de données nommée `BOUTIQUE`. On retient les informations suivantes :

- chaque client est identifié par un numéro et possède un nom, un numéro de téléphone et un quartier de résidence ;
- chaque vente possède un numéro, une date, un montant total et concerne un seul client ;
- un client peut effectuer plusieurs ventes, mais une vente n'appartient qu'à un seul client.

Travail à faire :

1. Définis les termes suivants : base de données, SGBD, clé primaire, clé étrangère.
2. Construis les tables `CLIENT` et `VENTE` en précisant leurs champs, la clé primaire de chacune (soulignée) et la clé étrangère (précédée de #).
3. Écris le schéma relationnel complet de la base `BOUTIQUE`.
4. Remplis la table `CLIENT` avec trois enregistrements fictifs de ton choix, puis la table `VENTE` avec quatre enregistrements cohérents (au moins deux ventes pour le même client).
5. Décris en langage naturel les étapes de la requête permettant d'ajouter une nouvelle vente d'un montant de 15000 FCFA pour le client numéro 2. Quelle vérification le SGBD doit-il effectuer avant d'accepter cette insertion ?
6. Décris en langage naturel la requête permettant de lister tous les achats (numéro de vente, date, montant) d'un client donné, par exemple le client numéro 2.
7. Le champ `telephone` de la table `CLIENT` est déclaré unique. Que se passe-t-il si deux clients donnent le même numéro de téléphone ? Quelle contrainte est alors mise en jeu ?

EXERCICE 18 Gestion des notes dans un lycée de Buea

Le conseil d'enseignement d'un lycée de Buea décide d'informatiser les notes des élèves de Première. La base de données `NOTES_LYCEE` contient les tables suivantes :

Table	Champs
CLASSE	<u>id_classe</u> , nom_classe, effectif
ELEVE	<u>matricule</u> , nom, sexe, date_naiss, #id_classe
MATIERE	<u>id_matiere</u> , intitule, coefficient
EVALUATION	<u>id_eval</u> , #matricule, #id_matiere, sequence, note

Travail à faire :

1. Pour chaque table, identifie la clé primaire et les éventuelles clés étrangères. Justifie le rôle de chaque clé étrangère.
2. Pourquoi la table `EVALUATION` est-elle indispensable ? Ne pourrait-on pas placer la note directement dans la table `ELEVE` ? Justifie ta réponse.
3. Classe les opérations suivantes selon qu'il s'agit d'une fonction de définition ou d'une fonction de manipulation : (a) créer la table `MATIERE` ; (b) saisir la note d'un élève en informatique ; (c) ajouter la colonne `appreciation` à la table `EVALUATION` ; (d) consulter les notes de la séquence 2 ; (e) supprimer la table `EVALUATION` ; (f) corriger une note mal saisie.
4. Le champ `note` doit être un nombre compris entre 0 et 20, et le champ `sequence` ne peut prendre que les valeurs 1 à 6. Pour chaque champ, cite la contrainte d'intégrité à mettre en place et explique ce qui se passe si l'utilisateur saisit la note 25 ou la séquence 8.
5. Décris en langage naturel la requête permettant d'afficher les noms de tous les élèves de la classe « Première C ».

EXERCICE 19 Centre de santé de Ngaoundéré

Le centre de santé intégré de Ngaoundéré veut gérer ses consultations à l'aide d'une base de données. On dispose des informations suivantes : chaque patient possède un code, un nom, un sexe, une date de naissance et un numéro de téléphone (facultatif) ; chaque médecin possède un matricule, un nom et une spécialité ; chaque consultation possède un numéro, une date, un diagnostic et concerne un patient et un médecin. Le tarif d'une consultation est fixé par défaut à 2000 FCFA.

Travail à faire :

1. Donne deux exemples de SGBD que le centre pourrait utiliser, en précisant un avantage de chacun.
2. Construis les tables `PATIENT`, `MEDECIN` et `CONSULTATION` en précisant les champs, les clés

primaires (soulignées) et les clés étrangères (précédées de #), puis écris le schéma relationnel de la base.

3. Pour chacun des champs suivants, indique la contrainte d'intégrité la plus adaptée et justifie ton choix : (a) le numéro de téléphone du patient ; (b) le tarif de la consultation ; (c) le sexe du patient ; (d) le code du patient ; (e) la date de la consultation.
4. Le réceptionniste tente d'enregistrer une consultation pour le patient de code P999, qui n'existe pas dans la table PATIENT. Le SGBD refuse l'opération : explique pourquoi et nomme le mécanisme mis en jeu.
5. Décris en langage naturel les requêtes permettant de : (a) insérer un nouveau médecin ; (b) afficher toutes les consultations du 15 mars 2025 ; (c) supprimer un patient qui n'a jamais eu de consultation.
6. Le directeur du centre souhaite ajouter le groupe sanguin des patients. S'agit-il d'une fonction de définition ou de manipulation ? Décris l'opération à réaliser.

12 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM de synthèse

Question 1 : réponse b). Un **SGBD** (Système de Gestion de Bases de Données) est un logiciel qui permet de créer, de décrire, de manipuler et d'administrer des **bases de données**. La réponse a) décrit la base de données elle-même, et non le logiciel qui la gère ; la réponse c) décrit un **enregistrement**.

Question 2 : réponse b). Un **enregistrement** est une ligne (ou tuple) d'une **table** ; il regroupe les valeurs des différents **champs** pour un même objet (par exemple un élève). Une colonne correspond à un champ.

Question 3 : réponse c). La **clé étrangère** est un champ d'une table qui reprend la valeur de la **clé primaire** d'une autre table afin d'établir un lien entre les deux tables (par exemple `#id_classe` dans `ELEVE` renvoie à la table `CLASSE`).

Question 4 : réponse b). **MySQL** est un SGBD libre très répandu. Microsoft Word est un logiciel de traitement de texte, Mozilla Firefox un navigateur web et Windows 11 un système d'exploitation.

Question 5 : réponse b). La **valeur nulle** (NULL) traduit l'absence de valeur (information inconnue ou non renseignée) ; la **valeur par défaut** est une valeur automatiquement attribuée par le SGBD lorsque l'utilisateur ne saisit rien. Une valeur nulle n'est ni zéro ni une chaîne vide.

CORRIGÉ Exercice 2 — Vrai ou faux, avec justification

1. Faux. La **clé primaire** identifie de manière unique chaque enregistrement d'une table : deux enregistrements ne peuvent jamais avoir la même valeur de clé primaire (contrainte d'unicité et de non-nullité).
2. Vrai. Une table peut être liée à plusieurs autres tables : elle contient alors une **clé étrangère** par lien. Par exemple, la table `EVALUATION` possède deux clés étrangères : `#matricule` (vers `ELEVE`) et `#id_matiere` (vers `MATIERE`).
3. Faux. Supprimer une colonne modifie la structure de la table : c'est une fonction de définition de la base de données (LDD). Les fonctions de manipulation (LMD) portent sur les données (insertion, mise à jour, suppression, consultation d'enregistrements).
4. Faux. La contrainte « **valeur nulle interdite** » (NOT NULL) impose précisément que le champ soit renseigné : le SGBD refusera toute saisie laissant ce champ vide.
5. Vrai. Microsoft Access, PostgreSQL et Oracle Database sont bien des **SGBD** : Access et Oracle sont propriétaires, PostgreSQL est libre.

CORRIGÉ Exercice 3 — Définitions

1. **Donnée** : élément brut (nombre, texte, date, image...) décrivant un fait ou une caractéristique d'un objet du monde réel. Exemple : la note « 15/20 » obtenue par un élève au devoir d'informatique.
2. **Base de données** : ensemble structuré et organisé de données relatives à un même domaine, stockées sur un support informatique et accessibles à plusieurs utilisateurs. Exemple : la base `GESTION_LYCEE` regroupant les élèves, les classes et les notes du lycée.
3. **Table** : structure d'une base de données relationnelle organisée en lignes et en colonnes, qui regroupe les informations concernant un même type d'objet. Exemple : la table `ELEVE`.
4. **Champ** : colonne d'une table, caractérisée par un nom et un type, qui représente une propriété commune à tous les enregistrements. Exemple : le champ `date_naiss` de la table `ELEVE`.
5. **Enregistrement** : ligne d'une table, contenant les valeurs des champs pour un objet précis. Exemple : la ligne (E001, Ngo Bell Amina, F, 12/03/2007, C01).
6. **Clé primaire** : champ (ou ensemble de champs) dont la valeur identifie de manière unique chaque enregistrement d'une table ; elle ne peut être ni nulle ni dupliquée. Exemple : le champ `matricule` de la table `ELEVE`.

CORRIGÉ Exercice 4 — Lecture directe d'une table

1. La table `ELEVE` contient 5 champs (`matricule`, `nom`, `sexe`, `date_naiss`, `#id_classe`) et 6 enregistrements (les six lignes de données, de E001 à E006).
2. La **clé primaire** est le champ `matricule` : ses valeurs (E001 à E006) sont toutes distinctes et non nulles, donc chaque enregistrement est identifié de manière unique.
3. Le champ `#id_classe` est une **clé étrangère** : il indique la classe à laquelle appartient chaque élève. Il renvoie probablement à la clé primaire `id_classe` d'une table `CLASSE` contenant la liste des classes du lycée (C01, C02, C03...).
4. Le champ `nom` ne peut pas être clé primaire car il n'est pas unique : on observe dans la table deux enregistrements portant le nom « Ngo Bell Amina » (E001 et E005, deux élèves différentes nées à des dates différentes). Or une clé primaire doit identifier chaque enregistrement de manière unique.

CORRIGÉ Exercice 5 — Définition ou manipulation ?

Rappel : les fonctions de définition (LDD) agissent sur la structure de la base (création/suppression de bases et de tables, ajout/renommage de colonnes, modification de type) ; les fonctions de manipulation (LMD) agissent sur les données (insertion, mise à jour, suppression, consultation d'enregistrements).

Fonction de définition (LDD)	Fonction de manipulation (LMD)
1. Créer la table <code>ELEVE</code>	2. Insérer un nouvel élève dans <code>ELEVE</code>
3. Ajouter la colonne <code>telephone_parent</code>	4. Modifier la note d'informatique d'une élève
5. Supprimer la base de données de l'année précédente	7. Consulter la liste des élèves absents
6. Renommer <code>date_naiss</code> en <code>naissance</code>	
8. Changer le type du champ <code>moyenne</code>	

Justifications : créer ou supprimer une table ou une base, ajouter ou renommer une colonne, changer un type modifient le schéma (la structure) de la base : ce sont des fonctions de définition. Insérer, modifier ou consulter des enregistrements agit sur le contenu : ce sont des fonctions de manipulation.

CORRIGÉ Exercice 6 — Quelle contrainte est violée ?

1. Contrainte de **clé primaire** (unicité). Le matricule est la clé primaire : deux enregistrements ne peuvent pas porter la même valeur `E001`, sinon l'identification unique des élèves serait impossible.
2. Contrainte de **valeur nulle interdite** (NOT NULL). Le champ `nom` est obligatoire : le SGBD refuse un enregistrement dont le nom est absent, car une valeur nulle y est interdite.
3. Contrainte de **format de valeur**. « 32/15/2020 » n'est pas une date valide (un mois ne dépasse pas 12, un jour ne dépasse pas 31) : la saisie ne respecte pas le format date attendu.
4. Contrainte de **valeur autorisée** (domaine de valeurs). Le champ `sexe` n'accepte que les valeurs « M » ou « F » ; « X » est hors du domaine autorisé.
5. Contrainte d' (clé étrangère). La valeur `C09` de la clé étrangère `#id_classe` n'existe pas comme clé primaire dans la table `CLASSE` : on ne peut pas rattacher un élève à une classe inexistante.
6. Contrainte d'**unicité de valeur** (UNIQUE). Le champ `téléphone` est déclaré unique : deux enregistrements ne peuvent pas contenir le même numéro « 699 00 00 00 ».

CORRIGÉ Exercice 7 — Schéma relationnel d'une bibliothèque scolaire

1. Tables et champs :

- LIVRE(code_livre, titre, auteur, annee_edition)
- EMPRUNTEUR(num_carte, nom, classe)
- PRET(num_pret, #num_carte, #code_livre, date_pret, date_retour_prevue)

2. Les **clés primaires** sont soulignées ci-dessus : code_livre pour LIVRE, num_carte pour EMPRUNTEUR, num_pret pour PRET. Les **clés étrangères**, précédées de #, sont #num_carte et #code_livre dans la table PRET.

3. Schéma relationnel complet :

```
LIVRE(code_livre, titre, auteur, annee_edition)
EMPRUNTEUR(num_carte, nom, classe)
PRET(num_pret, #num_carte, #code_livre, date_pret, date_retour_prevue)
-- #num_carte reference EMPRUNTEUR(num_carte)
-- #code_livre reference LIVRE(code_livre)
```

4. La table PRET contient deux clés étrangères car chaque prêt met en relation deux objets : un emprunteur (d'où #num_carte qui renvoie à EMPRUNTEUR) et un livre (d'où #code_livre qui renvoie à LIVRE). Ces deux clés étrangères matérialisent les liens entre le prêt, l'emprunteur et le livre, et garantissent qu'on ne prête qu'un livre existant à un emprunteur enregistré.

CORRIGÉ Exercice 8 — Composants et exemples de SGBD

1. Quatre composants d'un SGBD :

- le moteur de stockage (gestionnaire de fichiers) : il organise et gère physiquement le stockage des données sur le disque ;
- le langage de définition et de manipulation des données (LDD/LMD, par exemple **SQL**) : il permet de décrire la structure de la base et d'insérer, modifier, supprimer ou consulter les données ;
- le gestionnaire de sécurité et des droits d'accès : il contrôle l'authentification des utilisateurs et leurs autorisations (qui peut lire, écrire, supprimer) ;
- le dictionnaire des données (métabase) : il conserve la description de la structure de la base (tables, champs, types, contraintes).

(On pouvait aussi citer le gestionnaire de transactions et de concurrence, ou les outils de sauvegarde et de restauration.)

2. Quatre exemples de SGBD :

SGBD	Licence
MySQL	libre (avec une version commerciale)
PostgreSQL	libre
Oracle Database	propriétaire
Microsoft Access / SQL Server	propriétaire

3. Quatre domaines d'application au Cameroun :

- Banques (Afriland, BICEC...) : numéros de comptes, soldes, opérations de dépôt et de retrait ;
- Hôpitaux (hôpital central de Yaoundé...) : dossiers des patients, consultations, ordonnances ;
- Écoles et lycées : matricules des élèves, notes, bulletins, effectifs des classes ;
- Opérateurs de téléphonie mobile (MTN, Orange) : numéros d'abonnés, forfaits, historiques d'appels et de rechargements.

CORRIGÉ Exercice 9 — Facturation d'une boutique de Douala

Barème indicatif (sur 20) : question 1 : 4 pts ; question 2 : 4 pts ; question 3 : 2 pts ; question 4 : 4 pts ; question 5 : 2 pts ; question 6 : 2 pts ; question 7 : 2 pts.

1. Définitions :

- **Base de données** : ensemble structuré de données relatives à un même domaine, stockées sur support informatique et accessibles à plusieurs utilisateurs.
- **SGBD** : logiciel permettant de créer, de décrire, de manipuler et d'administrer des bases de données.
- **Clé primaire** : champ dont la valeur identifie de manière unique chaque enregistrement d'une table ; elle est non nulle et non dupliquée.
- **Clé étrangère** : champ d'une table qui reprend la valeur de la clé primaire d'une autre table afin d'établir un lien entre les deux tables.

2. Tables :

- CLIENT(num_client, nom, telephone, quartier)
- VENTE(num_vente, date_vente, montant, #num_client)

La clé étrangère #num_client dans VENTE traduit le fait qu'une vente appartient à un seul client, tandis qu'un client peut effectuer plusieurs ventes (relation 1 : n).

3. Schéma relationnel de la base BOUTIQUE :

CLIENT(num_client, nom, telephone, quartier)


```
VENTE(num_vente, date_vente, montant, #num_client)
```

```
-- #num_client reference CLIENT(num_client)
```

4. Remplissage (exemple) :

Table CLIENT :

num_client	nom	telephone	quartier
1	Kamga Brice	690 11 22 33	Akwa
2	Tchoupo Marlyse	699 44 55 66	Bonabéri
3	Ndi Charles	677 88 99 00	Deïdo

Table VENTE :

num_vente	date_vente	montant (FCFA)	#num_client
V1	02/09/2024	5000	1
V2	03/09/2024	12500	2
V3	05/09/2024	3000	2
V4	06/09/2024	8750	3

Le client numéro 2 apparaît bien dans deux ventes (V2 et V3), et chaque vente ne concerne qu'un seul client.

- Ajout d'une vente de 15000 FCFA pour le client 2 : on choisit un nouveau numéro de vente (par exemple V5), on saisit la date du jour, le montant 15000 et la valeur 2 pour #num_client, puis on insère l'enregistrement dans VENTE. Avant d'accepter, le SGBD vérifie l'intégrité référentielle : la valeur 2 doit exister comme clé primaire dans la table CLIENT (sinon l'insertion est refusée). Il vérifie aussi que le nouveau numéro de vente n'existe pas déjà (unicité de la clé primaire).
- Liste des achats du client 2 : on parcourt la table VENTE, on sélectionne les enregistrements dont le champ #num_client vaut 2, puis on affiche pour chacun le numéro de vente, la date et le montant. Résultat attendu : (V2, 03/09/2024, 12500) et (V3, 05/09/2024, 3000).
- Si deux clients donnent le même numéro de téléphone alors que le champ est déclaré unique, le SGBD refuse la deuxième saisie et affiche une erreur : c'est la contrainte d'unicité de valeur (UNIQUE) qui est mise en jeu. Le deuxième client devra fournir un autre numéro (ou laisser le champ vide si la valeur nulle est autorisée).

Points de vigilance : exiger la justification du placement de la clé étrangère du côté « plusieurs » (dans VENTE) ; vérifier la cohérence des valeurs de #num_client avec la table CLIENT ; citer explicitement l'intégrité référentielle à la question 5.

CORRIGÉ Exercice 10 — Gestion des notes dans un lycée de Buea

Barème indicatif (sur 20) : question 1 : 5 pts ; question 2 : 4 pts ; question 3 : 3 pts ; question 4 : 4 pts ; question 5 : 4 pts.

1. Clés de chaque table :

- CLASSE : clé primaire `id_classe` ; pas de clé étrangère.
- ELEVE : clé primaire `matricule` ; clé étrangère `#id_classe`, qui rattache chaque élève à sa classe (elle renvoie à CLASSE).
- MATIERE : clé primaire `id_matiere` ; pas de clé étrangère.
- EVALUATION : clé primaire `id_eval` ; deux clés étrangères : `#matricule` (renvoie à ELEVE, indique quel élève est évalué) et `#id_matiere` (renvoie à MATIERE, indique dans quelle matière la note a été obtenue).

2. La table EVALUATION est indispensable car un élève obtient plusieurs notes (plusieurs matières, plusieurs séquences) et une matière concerne plusieurs élèves : il s'agit d'une relation « plusieurs à plusieurs » entre ELEVE et MATIERE, qui se traduit par une table intermédiaire. Placer la note directement dans ELEVE obligerait soit à n'enregistrer qu'une seule note par élève, soit à répéter les informations de l'élève autant de fois qu'il a de notes, ce qui créerait des redondances et des risques d'incohérence.

3. Classification :

- Fonctions de définition (LDD) : (a) créer la table MATIERE ; (c) ajouter la colonne `appreciation` ; (e) supprimer la table EVALUATION.
- Fonctions de manipulation (LMD) : (b) saisir la note d'un élève (insertion) ; (d) consulter les notes de la séquence 2 (consultation) ; (f) corriger une note mal saisie (mise à jour).

4. Pour le champ `note` : contrainte de valeur autorisée (domaine), exprimée par la condition $0 \leq \text{note} \leq 20$. Si l'utilisateur saisit 25, le SGBD refuse la saisie et signale une erreur, car 25 est hors de l'intervalle autorisé. Pour le champ `sequence` : contrainte de valeur autorisée limitée à l'ensemble $\{1, 2, 3, 4, 5, 6\}$. La saisie de la séquence 8 est refusée car 8 n'appartient pas à cet ensemble.

5. Requête en langage naturel : on sélectionne dans la table CLASSE l'enregistrement dont `nom_classe` vaut « Première C » et on retient son `id_classe` ; puis on parcourt la table ELEVE en ne gardant que les enregistrements dont le champ `#id_classe` est égal à cet identifiant ; on affiche enfin le champ `nom` de ces enregistrements.

Points de vigilance : la justification de la question 2 doit mentionner la relation « plusieurs à plusieurs » et le problème de redondance ; à la question 4, le nom exact de la contrainte (valeur autorisée / domaine de valeurs) est attendu, pas seulement « erreur de saisie ».

CORRIGÉ Exercice 11 — Centre de santé de Ngaoundéré

Barème indicatif (sur 20) : question 1 : 2 pts ; question 2 : 5 pts ; question 3 : 5 pts ; question 4 : 2 pts ; question 5 : 4 pts ; question 6 : 2 pts.

1. Deux SGBD possibles :

- MySQL (libre) : gratuit, fiable et largement documenté, ce qui réduit les coûts du centre ;
- Microsoft Access (propriétaire) : interface graphique simple, adaptée à un petit centre sans informaticien spécialisé.

(On pouvait aussi citer PostgreSQL, libre et très robuste, ou Oracle Database, propriétaire et très performant mais coûteux.)

2. Tables et schéma relationnel :

- PATIENT(code_patient, nom, sexe, date_naiss, telephone)
- MEDECIN(matricule_med, nom, specialite)
- CONSULTATION(num_consult, date_consult, diagnostic, tarif, #code_patient, #matricule_med)

```
PATIENT(code_patient, nom, sexe, date_naiss, telephone)
MEDECIN(matricule_med, nom, specialite)
CONSULTATION(num_consult, date_consult, diagnostic, tarif,
              #code_patient, #matricule_med)
-- #code_patient reference PATIENT(code_patient)
-- #matricule_med reference MEDECIN(matricule_med)
```

3. Contraintes adaptées :

- (a) telephone : valeur nulle autorisée, car le numéro est facultatif (on peut éventuellement ajouter une contrainte d'unicité) ;
- (b) tarif : valeur par défaut de 2000 FCFA, attribuée automatiquement si le réceptionniste ne saisit rien ;
- (c) sexe : valeur autorisée limitée à « M » ou « F » ;
- (d) code_patient : contrainte de clé primaire (unicité et non-nullité), car il identifie chaque patient de manière unique ;
- (e) date_consult : format de valeur de type date (jour/mois/année valide), éventuellement complétée par « valeur nulle interdite ».

4. Le SGBD refuse l'opération parce que la valeur P999 de la clé étrangère #code_patient n'existe pas comme clé primaire dans la table PATIENT : on ne peut pas enregistrer une consultation pour un patient inconnu. Le mécanisme mis en jeu est la contrainte d'intégrité référentielle (liée à la clé étrangère).

5. Requêtes en langage naturel :

- (a) Insérer un médecin : on saisit un matricule inexistant, le nom et la spécialité, puis on ajoute l'enregistrement dans `MEDECIN` ; le SGBD vérifie l'unicité du matricule.
 - (b) Consultations du 15 mars 2025 : on parcourt la table `CONSULTATION` et on sélectionne les enregistrements dont `date_consult` est égale au 15/03/2025, puis on affiche leurs informations.
 - (c) Supprimer un patient sans consultation : on vérifie d'abord dans `CONSULTATION` qu'aucun enregistrement ne possède `#code_patient` égal au code de ce patient ; si c'est le cas, on supprime l'enregistrement correspondant dans `PATIENT`. (Cette vérification préserve l'intégrité référentielle.)
6. Ajouter le groupe sanguin des patients consiste à ajouter une colonne `groupe_sanguin` à la table `PATIENT` : c'est une modification de la structure de la base, donc une fonction de définition (LDD : `ALTER TABLE`). On peut compléter cette colonne par une contrainte de valeur autorisée (O+, O-, A+, A-, B+, B-, AB+, AB-).

Points de vigilance : à la question 3, exiger le nom de la contrainte accompagné d'une justification liée à l'énoncé (« facultatif » ⇒ valeur nulle autorisée ; « par défaut » ⇒ valeur par défaut) ; à la question 5(c), la vérification préalable de l'absence de consultations est un point clé du raisonnement.

13 FICHE BILAN

Carte mentale de la leçon : les six idées forces des bases de données relationnelles.

FICHE BILAN L'essentiel à connaître pour l'examen**1. Définitions clés à connaître par cœur**

- **Donnée** : élément brut décrivant un fait, un objet ou une personne (ex. : le nom « Ngo Bell », la note 15).
- **Base de données (BD)** : ensemble structuré et organisé de données reliées entre elles, stockées de façon durable et partageable.
- **SGBD** (Système de Gestion de Bases de Données) : logiciel qui permet de créer, décrire, manipuler et protéger une base de données. Exemples : MySQL, PostgreSQL, Oracle, Microsoft Access, SQLite.
- **Table** (ou relation) : structure en lignes et colonnes qui regroupe les données relatives

à un même type d'objet (ex. : table `ELEVE`).

- **Champ** (colonne) : propriété élémentaire d'une table, caractérisée par un nom et un type (texte, nombre, date...).
- **Enregistrement** (ligne ou n-uplet) : occurrence d'une table, c'est-à-dire l'ensemble des valeurs décrivant un objet précis.
- **Clé primaire** : champ (ou groupe de champs) qui identifie de façon unique chaque enregistrement d'une table ; jamais nulle, jamais dupliquée.
- **Clé étrangère** : champ d'une table qui fait référence à la clé primaire d'une autre table, créant ainsi un lien entre les tables.

2. Les deux grandes familles de fonctions d'un SGBD

Famille	Rôle	Opérations typiques
Définition (LDD)	Décrire la structure de la BD	créer/supprimer une BD ou une table ; ajouter ou renommer une colonne ; modifier le type d'un champ
Manipulation (LMD)	Agir sur le contenu de la BD	insertion, consultation (recherche), mise à jour, suppression d'enregistrements

3. Les contraintes d'intégrité (garantes de la qualité des données)

- **Contrainte de clé primaire** : identifiant unique et non nul pour chaque ligne.
- **Contrainte d'unicité** : deux enregistrements ne peuvent avoir la même valeur (ex. : numéro de matricule).
- **Contrainte de valeur nulle** : impose (ou interdit) qu'un champ reste vide.
- **Valeur par défaut** : valeur affectée automatiquement si l'utilisateur n'en saisit pas (ex. : ville = « Yaoundé »).
- **Valeur autorisée** (domaine) : la valeur doit appartenir à un ensemble ou intervalle donné (ex. : $0 \leq \text{note} \leq 20$).
- **Format de valeur** : la valeur doit respecter un gabarit (ex. : date JJ/MM/AAAA, téléphone à 9 chiffres).

4. Méthodes express

- Identifier la clé primaire d'une table : chercher le champ dont la valeur est unique pour chaque ligne et jamais vide ; à défaut, ajouter un identifiant (numéro).
- Repérer une clé étrangère : c'est un champ qui porte le même nom (et les mêmes valeurs) que la clé primaire d'une autre table.
- Choisir la bonne opération : on touche à la structure $\Rightarrow$ fonction de définition ; on touche aux données $\Rightarrow$ fonction de manipulation.

5. Domaines d'application : gestion des élèves et des bulletins au lycée, bibliothèque scolaire, fiches de paie, stocks d'une boutique, réservations, services bancaires (mobile money), administrations en ligne.

À RETENIR Checklist avant le Bac

Avant l'épreuve, vérifie que tu sais :

- ☐ Définir précisément : donnée, base de données, SGBD, table, champ, enregistrement.
- ☐ Citer au moins trois exemples de SGBD (MySQL, Oracle, Access, PostgreSQL, SQLite).
- ☐ Citer au moins trois domaines d'application des bases de données au Cameroun.
- ☐ Décrire les composants d'un SGBD (moteur, dictionnaire de données, interfaces/langages, outils d'administration).
- ☐ Distinguer clairement champ (colonne) et enregistrement (ligne) dans une table donnée.
- ☐ Identifier la clé primaire d'une table et justifier ton choix (unicité + non nullité).
- ☐ Repérer une clé étrangère et expliquer le lien qu'elle crée entre deux tables.
- ☐ Énumérer les fonctions de définition : création/suppression de BD et de tables, ajout/renomage de colonnes, modification du type d'un champ.
- ☐ Énumérer les fonctions de manipulation : insertion, consultation, mise à jour, suppression.
- ☐ Classer une opération donnée en « définition » ou « manipulation » sans hésiter.
- ☐ Décrire les six contraintes d'intégrité : clé primaire, unicité, valeur nulle, valeur par défaut, valeur autorisée, format.
- ☐ Donner un exemple concret de chaque contrainte (note entre 0 et 20, matricule unique, ville par défaut...).



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+